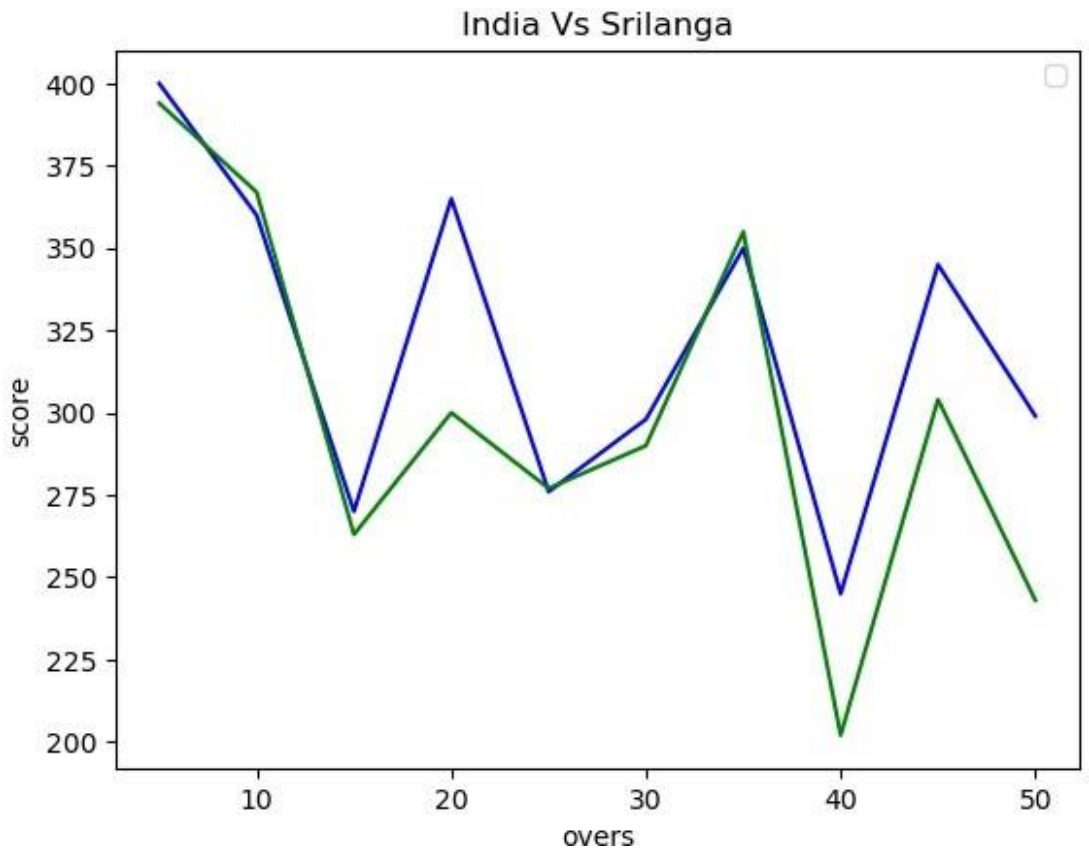
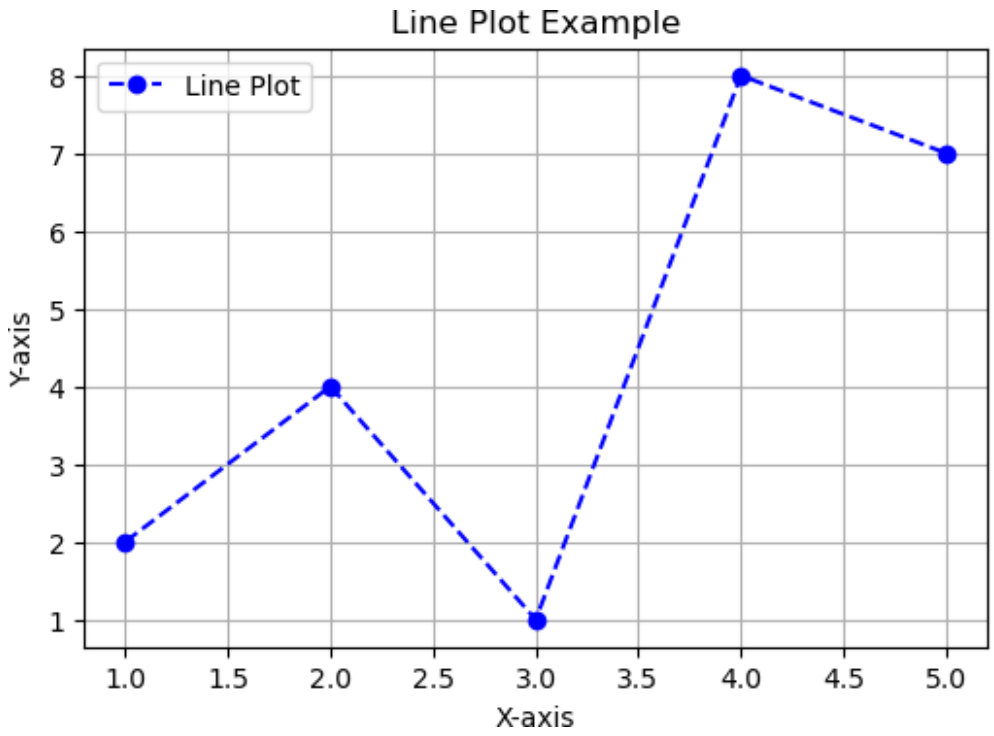
# EXERCISE 1

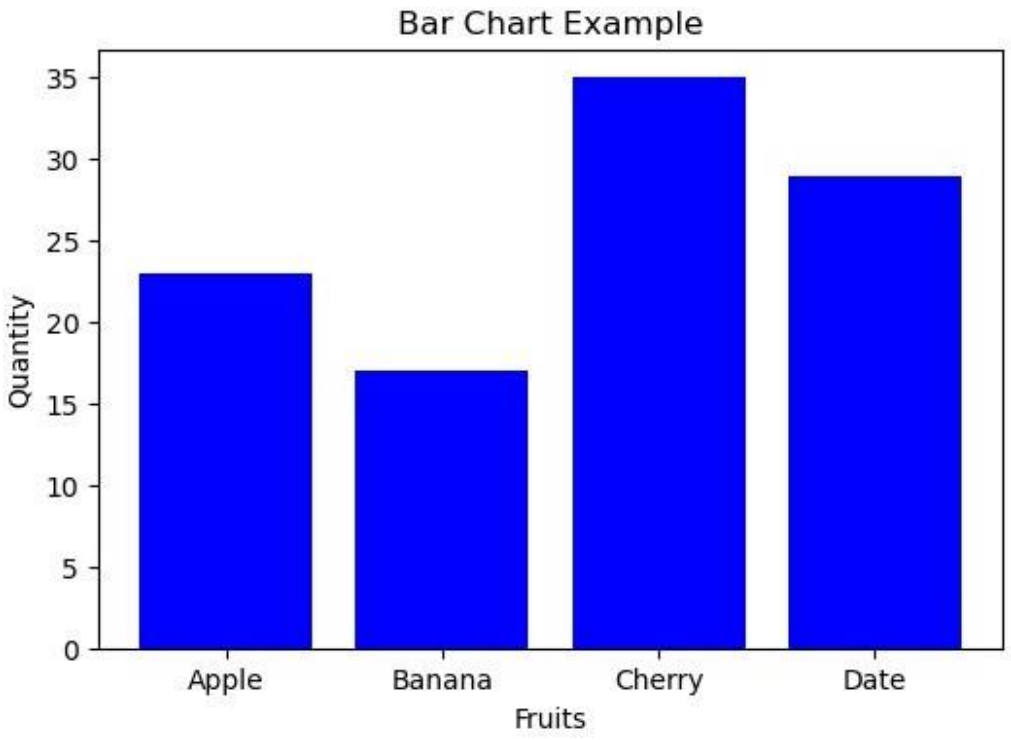
```
#sadha
#240701528
#22.7.25
#LinePlot1
import matplotlib.pyplot as plt
overs=list(range(5,51,5))
India_Score=[400,360,270,365,276,298,350,245,345,299]
Srilanga_Score=[394,367,263,300,277,290,355,202,304,243]
plt.plot(overs,India_Score,'color'=='blue')
plt.plot(overs,Srilanga_Score)
plt.show()
plt.title("India Vs Srilanga")
plt.xlabel("overs")
plt.ylabel("score")
plt.legend()
plt.plot(overs,India_Score,color="blue",label="India")
plt.plot(overs,Srilanga_Score,color="green",label="Srilanga")
```



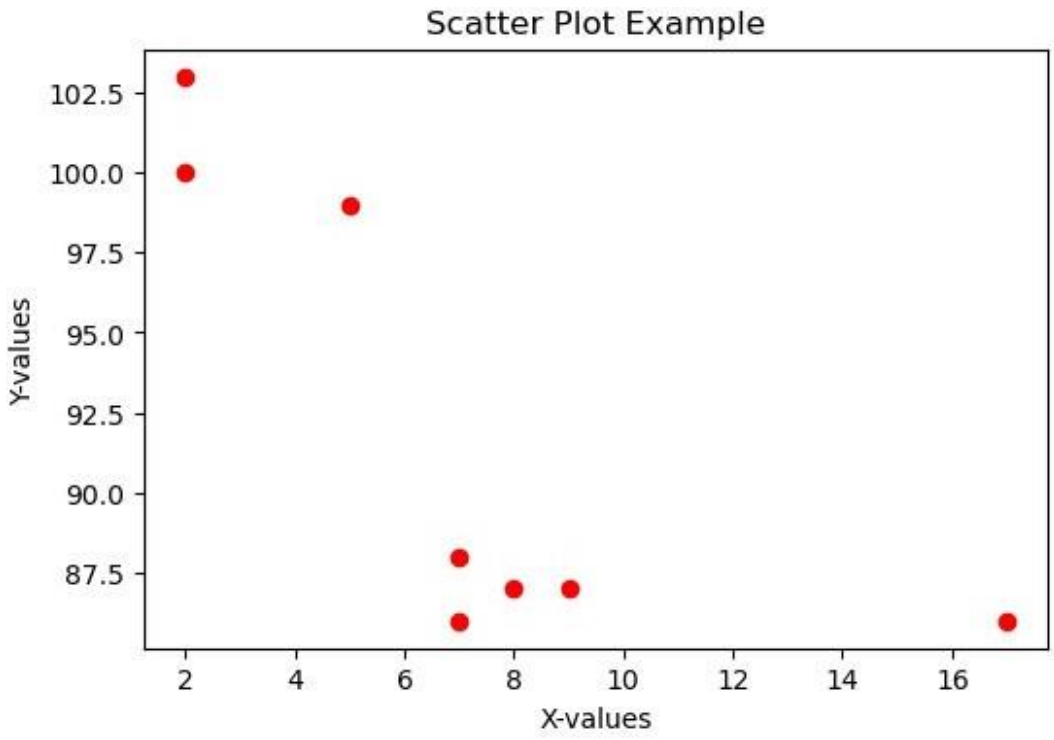

```
#sadha
#240701528
#22.7.25
#LinePlot2
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [2, 4, 1, 8, 7]
plt.figure(figsize=(6, 4)) # Set the figure size
plt.plot(x, y, color='blue', marker='o', linestyle='--',label='Line
Plot')
plt.title("Line Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend()
plt.grid(True)
plt.show()
```



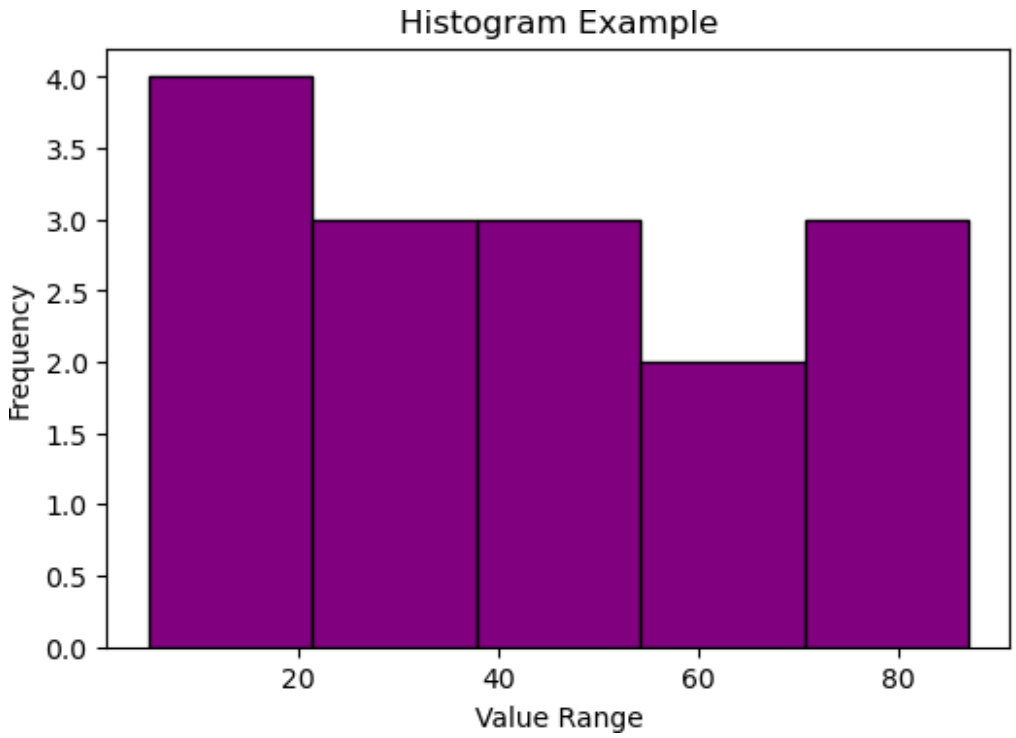

```
#sadha
#240701528
#22.7.25
#BarChart
categories = ['Apple','Banana','Cherry','Date']
values = [23, 17, 35, 29]
plt.figure(figsize=(6, 4))
plt.bar(categories,values,color='blue')
plt.title("Bar Chart Example")
plt.xlabel("Fruits")
plt.ylabel("Quantity")
plt.show()
```



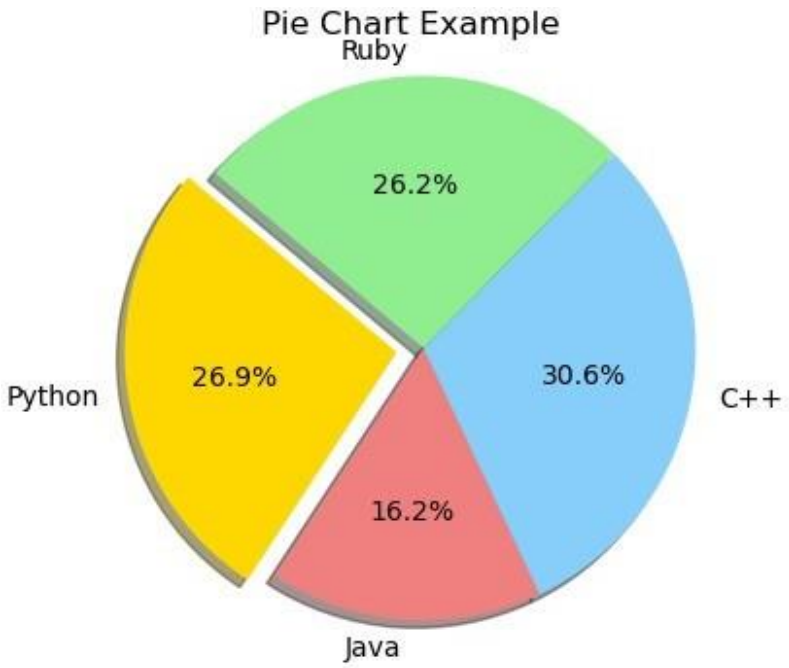

```
#sadha
#240701528
#22.7.25
#ScatterPlot
x_scatter = [5, 7, 8, 7, 2, 17, 2, 9]
y_scatter = [99, 86, 87, 88, 100, 86, 103, 87]
plt.figure(figsize=(6, 4))
plt.scatter(x_scatter,y_scatter,color='red')
plt.title("Scatter Plot Example")
plt.xlabel("X-values")
plt.ylabel("Y-values")
plt.show()
```




```
#sadha
#240701528
#22.7.25
#Histogram
data = [22, 87, 5, 43, 56, 73, 55, 54, 11, 20, 51, 5, 79, 31, 27]
plt.figure(figsize=(6, 4))
plt.hist(data, bins=5,color='purple',edgecolor='black')
plt.title("Histogram Example")
plt.xlabel("Value Range")
plt.ylabel("Frequency")
plt.show()
```



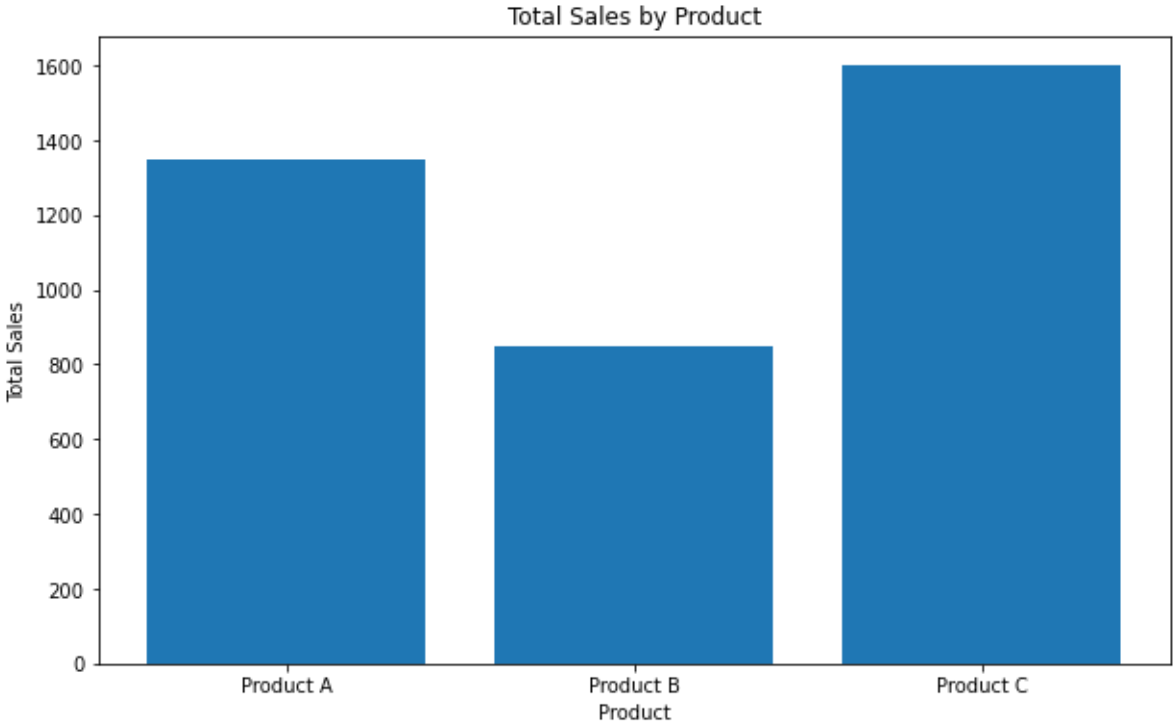

```
#sadha
#240701528
#22.7.25
#PieChart
labels = ['Python','Java','C++','Ruby']
sizes = [215, 130, 245, 210]
colors = ['gold','lightcoral','lightskyblue','lightgreen']
explode = (0.1, 0, 0, 0)
plt.figure(figsize=(6, 4))
plt.pie(sizes, explode=explode, labels=labels,
colors=colors,autopct='%1.1f%%',
shadow=True, startangle=140)
plt.title("Pie Chart Example")
plt.axis('equal')
plt.show()
```


Pie Chart Example
Ruby
26.2%
Python
26.9%
30.6%
C++
16.2%
Java

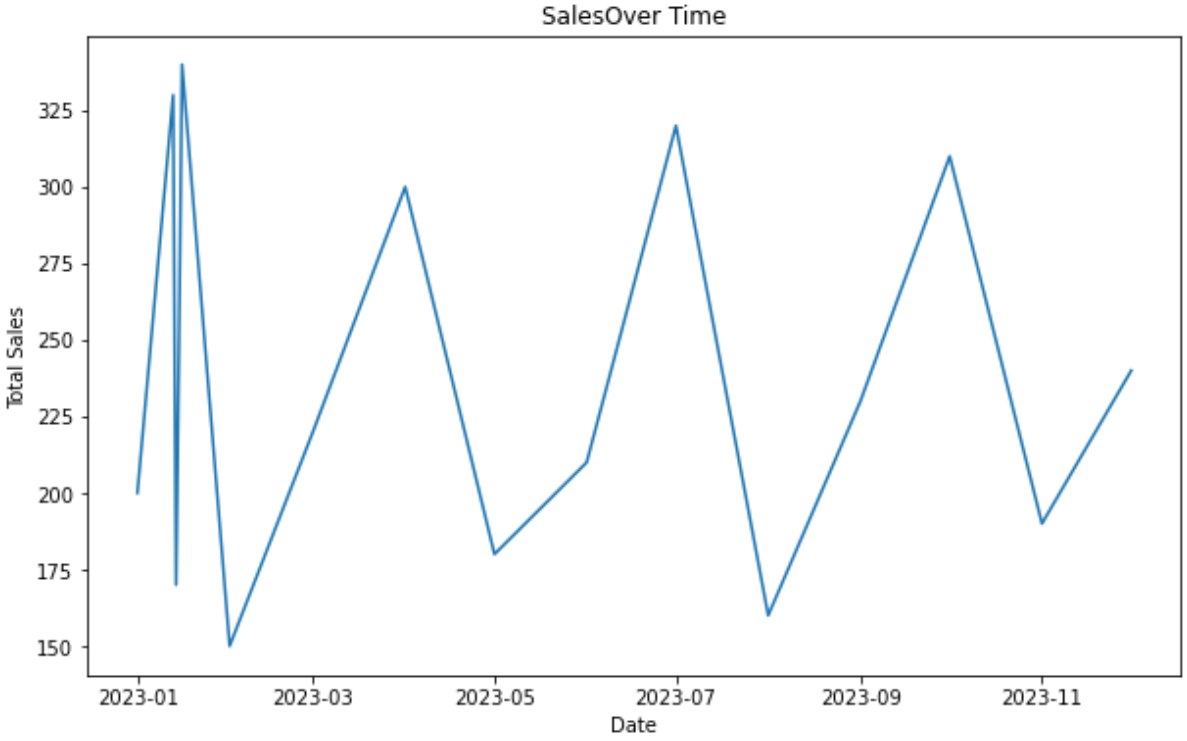
# EXERCISE 2

In [15]:
```
#sadha
#240701528
#29.7.25
#data preprocessing and and visualization
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
file_path='sales_data.csv'
df=pd.read_csv(file_path)
print(df.head())
print(df.isnull().sum())
df['Sales'].fillna(df['Sales'].mean(),  inplace=True)
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
print(df.describe())
product_summary = df.groupby('Product').agg({
'Sales': 'sum',
'Quantity': 'sum'
}).reset_index()
print(product_summary)
plt.figure(figsize=(10,  6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total  Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'],sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total  Sales')
plt.title('SalesOver Time')
plt.show()
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
aggfunc=np.sum, fill_value=0)
print(pivot_table)
correlation_matrix = df.corr()
print(correlation_matrix)
import seaborn as sns
plt.figure(figsize=(8,  6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000
     Product   Sales  Quantity
0  Product A    1350        33
1  Product B     850        17
2  Product C    1600        36
```

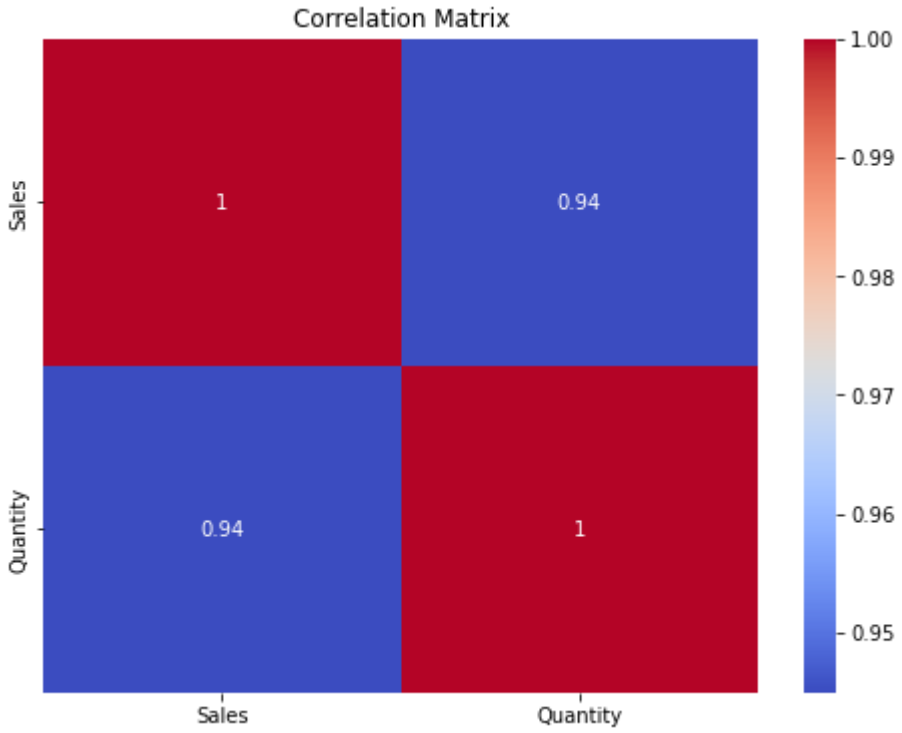


```
C:\Users\hdc0422189\AppData\Local\Temp\ipykernel_2780\302693546.py:26: UserWarnin
g: Parsing '13-01-2023' in DD/MM/YYYY format. Provide format or specify infer_date
time_format=True for consistent parsing.
  df['Date'] = pd.to_datetime(df['Date'])
C:\Users\hdc0422189\AppData\Local\Temp\ipykernel_2780\302693546.py:26: UserWarnin
g: Parsing '14-01-2023' in DD/MM/YYYY format. Provide format or specify infer_date
time_format=True for consistent parsing.
  df['Date'] = pd.to_datetime(df['Date'])
C:\Users\hdc0422189\AppData\Local\Temp\ipykernel_2780\302693546.py:26: UserWarnin
g: Parsing '15-01-2023' in DD/MM/YYYY format. Provide format or specify infer_date
time_format=True for consistent parsing.
  df['Date'] = pd.to_datetime(df['Date'])
C:\Users\hdc0422189\AppData\Local\Temp\ipykernel_2780\302693546.py:26: UserWarnin
g: Parsing '16-01-2023' in DD/MM/YYYY format. Provide format or specify infer_date
time_format=True for consistent parsing.
  df['Date'] = pd.to_datetime(df['Date'])
```




| Product | Product A | Product B | Product C |
|---|---|---|---|
| Region | | | |
| East | 0 | 0 | 1600 |
| North | 1350 | 0 | 0 |
| South | 0 | 480 | 0 |
| West | 0 | 370 | 0 |

| | Sales | Quantity |
|---|---|---|
| Sales | 1.000000 | 0.944922 |
| Quantity | 0.944922 | 1.000000 |


Correlation Matrix
Sales
Quantity
1
0.94
0.94
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95

In [ ]:

# EXERCISE 3

```
#sadha
#240701528
#5.8.25
#DataPrepocessing
import numpy as np
import pandas as pd
df=pd.read_csv("C:\\Users\\sri sadha\\Downloads\
pre_process_datasample.csv")
df
```

| | Country | Age | Salary | Purchased |
|---|---|---|---|---|
| 0 | France | 44.0 | 72000.0 | No |
| 1 | Spain | 27.0 | 48000.0 | Yes |
| 2 | Germany | 30.0 | 54000.0 | No |
| 3 | Spain | 38.0 | 61000.0 | No |
| 4 | Germany | 40.0 | NaN | Yes |
| 5 | France | 35.0 | 58000.0 | Yes |
| 6 | Spain | NaN | 52000.0 | No |
| 7 | France | 48.0 | 79000.0 | Yes |
| 8 | Germany | 50.0 | 83000.0 | No |
| 9 | France | 37.0 | 67000.0 | Yes |

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Country     10 non-null     object
 1   Age         9 non-null      float64
 2   Salary      9 non-null      float64
 3   Purchased   10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
df.Country.mode()
```

```
0    France
Name: Country, dtype: object
```

```
df.Country.mode()[0]
```

```
'France'
```

```
type(df.Country.mode())
```

```
pandas.core.series.Series
```

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
df.Age.fillna(df.Age.median(),inplace=True)
```

```
df.Salary.fillna(round(df.Salary.mean()),inplace=True)
df
```

```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_53620\1020198583.py:1:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
  df.Country.fillna(df.Country.mode()[0],inplace=True)
```

```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_53620\1020198583.py:2:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
  df.Age.fillna(df.Age.median(),inplace=True)
```

```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_53620\1020198583.py:3:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.
```

```
  df.Salary.fillna(round(df.Salary.mean()),inplace=True)
```

| | Country | Age | Salary | Purchased |
|---|---|---|---|---|
| 0 | France | 44.0 | 72000.0 | No |
| 1 | Spain | 27.0 | 48000.0 | Yes |
| 2 | Germany | 30.0 | 54000.0 | No |
| 3 | Spain | 38.0 | 61000.0 | No |

```
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

```
pd.get_dummies(df.Country)
```

```
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8   False     True  False
9    True    False  False
```

```
updated_dataset=pd.concat([pd.get_dummies(df.Country),df.iloc[:,
[1,2,3]]],axis=1)
updated_dataset
```

```
    France  Germany  Spain   Age   Salary Purchased
0     True    False  False  44.0  72000.0        No
1    False    False   True  27.0  48000.0       Yes
2    False     True  False  30.0  54000.0        No
3    False    False   True  38.0  61000.0        No
4    False     True  False  40.0  63778.0       Yes
5     True    False  False  35.0  58000.0       Yes
6    False    False   True  38.0  52000.0        No
7     True    False  False  48.0  79000.0       Yes
8    False     True  False  50.0  83000.0        No
9     True    False  False  37.0  67000.0       Yes
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        10 non-null     float64
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 452.0+ bytes
```

```
updated_dataset
```

| | France | Germany | Spain | Age | Salary | Purchased |
|---|---|---|---|---|---|---|
| 0 | True | False | False | 44.0 | 72000.0 | No |
| 1 | False | False | True | 27.0 | 48000.0 | Yes |
| 2 | False | True | False | 30.0 | 54000.0 | No |
| 3 | False | False | True | 38.0 | 61000.0 | No |
| 4 | False | True | False | 40.0 | 63778.0 | Yes |
| 5 | True | False | False | 35.0 | 58000.0 | Yes |
| 6 | False | False | True | 38.0 | 52000.0 | No |
| 7 | True | False | False | 48.0 | 79000.0 | Yes |
| 8 | False | True | False | 50.0 | 83000.0 | No |
| 9 | True | False | False | 37.0 | 67000.0 | Yes |

# EXERCISE 4

```
#sadha
#240701528
#5.8.25
#HandlingMissingData

import numpy as np
import pandas as pd
df=pd.read_csv("Hotel_Dataset.csv")
df
```

```
     CustomerID Age_Group   Rating(1-5)       Hotel FoodPreference   Bill
\
0             1      20-25             4        Ibis             veg   1300

1             2      30-35             5   LemonTree         Non-Veg   2000

2             3      25-30             6     RedFox             Veg   1322

3             4      20-25            -1   LemonTree             Veg   1234

4             5        35+             3        Ibis      Vegetarian    989

5             6        35+             3        Ibys         Non-Veg   1909

6             7        35+             4     RedFox      Vegetarian   1000

7             8      20-25             7   LemonTree             Veg   2999

8             9      25-30             2        Ibis         Non-Veg   3456

9             9      25-30             2        Ibis         Non-Veg   3456

10           10      30-35             5     RedFox         non-Veg  -6755


    NoOfPax  EstimatedSalary Age Group.1
0         2            40000       20-25
1         3            59000       30-35
2         2            30000       25-30
3         2           120000       20-25
4         2            45000         35+
5         2           122220         35+
6        -1            21122         35+
7       -10           345673       20-25
8         3           -99999       25-30
9         3           -99999       25-30
10        4            87777       30-35
```

```
df.duplicated()
```

```
0      False
1      False
2      False
3      False
4      False
5      False
6      False
7      False
8      False
9       True
10     False
dtype: bool
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   CustomerID        11 non-null     int64 
 1   Age_Group         11 non-null     object
 2   Rating(1-5)       11 non-null     int64 
 3   Hotel             11 non-null     object
 4   FoodPreference    11 non-null     object
 5   Bill              11 non-null     int64 
 6   NoOfPax           11 non-null     int64 
 7   EstimatedSalary   11 non-null     int64 
 8   Age_Group.1       11 non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes
```

```
df.drop_duplicates(inplace=True)
df
```

```
    CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference  Bill
\
0            1     20-25            4       Ibis            veg  1300

1            2     30-35            5  LemonTree        Non-Veg  2000

2            3     25-30            6     RedFox            Veg  1322

3            4     20-25           -1  LemonTree            Veg  1234

4            5       35+            3       Ibis     Vegetarian   989

5            6       35+            3       Ibys        Non-Veg  1909

6            7       35+            4     RedFox     Vegetarian  1000
```

```
7             8     20-25             7  LemonTree            Veg  2999

8             9     25-30             2       Ibis        Non-Veg  3456

10           10     30-35             5     RedFox        non-Veg -6755
```

```
    NoOfPax  EstimatedSalary Age_Group.1
0         2            40000       20-25
1         3            59000       30-35
2         2            30000       25-30
3         2           120000       20-25
4         2            45000         35+
5         2           122220         35+
6        -1            21122         35+
7       -10           345673       20-25
8         3           -99999       25-30
10        4            87777       30-35
```

```
len(df)
```

```
10
```

```
index=np.array(list(range(0,len(df))))
df.set_index(index,inplace=True)
index
```

```
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
df
```

```
   CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference  Bill
NoOfPax  \
0           1     20-25            4       Ibis            veg  1300
2
1           2     30-35            5  LemonTree        Non-Veg  2000
3
2           3     25-30            6     RedFox            Veg  1322
2
3           4     20-25           -1  LemonTree            Veg  1234
2
4           5       35+            3       Ibis     Vegetarian   989
2
5           6       35+            3       Ibys        Non-Veg  1909
2
6           7       35+            4     RedFox     Vegetarian  1000
-1
7           8     20-25            7  LemonTree            Veg  2999
-10
8           9     25-30            2       Ibis        Non-Veg  3456
3
```

```
9           10     30-35              5     RedFox         non-Veg -6755
4

   EstimatedSalary  Age_Group.1
0            40000        20-25
1            59000        30-35
2            30000        25-30
3           120000        20-25
4            45000          35+
5           122220          35+
6            21122          35+
7           345673        20-25
8           -99999        25-30
9            87777        30-35
```

```
df.drop(['Age_Group.1'],axis=1,inplace=True)
df
```

```
   CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference  Bill
NoOfPax  \
0           1     20-25             4       Ibis             veg  1300
2
1           2     30-35             5  LemonTree         Non-Veg  2000
3
2           3     25-30             6     RedFox             Veg  1322
2
3           4     20-25            -1  LemonTree             Veg  1234
2
4           5       35+             3       Ibis     Vegetarian   989
2
5           6       35+             3       Ibys         Non-Veg  1909
2
6           7       35+             4     RedFox     Vegetarian  1000
-1
7           8     20-25             7  LemonTree             Veg  2999
-10
8           9     25-30             2       Ibis         Non-Veg  3456
3
9          10     30-35             5     RedFox         non-Veg -6755
4

   EstimatedSalary
0            40000
1            59000
2            30000
3           120000
4            45000
5           122220
6            21122
7           345673
```

```
8           -99999
9            87777

df.CustomerID.loc[df.CustomerID<0]=np.nan
df.Bill.loc[df.Bill<0]=np.nan
df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
df

C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2080958306.py:1:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.CustomerID.loc[df.CustomerID<0]=np.nan
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2080958306.py:1:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.CustomerID.loc[df.CustomerID<0]=np.nan
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2080958306.py:2:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value
```

```
Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.Bill.loc[df.Bill<0]=np.nan
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2080958306.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.Bill.loc[df.Bill<0]=np.nan
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2080958306.py:3:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2080958306.py:3:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.EstimatedSalary.loc[df.EstimatedSalary<0]=np.nan
```

```
   CustomerID Age_Group  Rating(1-5)       Hotel FoodPreference    Bill
\
0         1.0     20-25            4       Ibis             veg  1300.0

1         2.0     30-35            5  LemonTree        Non-Veg  2000.0

2         3.0     25-30            6     RedFox             Veg  1322.0

3         4.0     20-25           -1  LemonTree             Veg  1234.0

4         5.0       35+            3       Ibis     Vegetarian   989.0

5         6.0       35+            3       Ibys        Non-Veg  1909.0

6         7.0       35+            4     RedFox     Vegetarian  1000.0

7         8.0     20-25            7  LemonTree             Veg  2999.0

8         9.0     25-30            2       Ibis        Non-Veg  3456.0

9        10.0     30-35            5     RedFox        non-Veg     NaN


   NoOfPax  EstimatedSalary
0        2          40000.0
1        3          59000.0
2        2          30000.0
3        2         120000.0
4        2          45000.0
5        2         122220.0
6       -1          21122.0
7      -10         345673.0
8        3              NaN
9        4          87777.0
```

```
df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
df
```

```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2129877948.py:1:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0!
You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy.
A typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value
```

```
Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\2129877948.py:1:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df['NoOfPax'].loc[(df['NoOfPax']<1) | (df['NoOfPax']>20)]=np.nan
```

| | CustomerID | Age_Group | Rating(1-5) | Hotel | FoodPreference | Bill |
|---|---|---|---|---|---|---|
| 0 | 1.0 | 20-25 | 4 | Ibis | veg | 1300.0 |
| 1 | 2.0 | 30-35 | 5 | LemonTree | Non-Veg | 2000.0 |
| 2 | 3.0 | 25-30 | 6 | RedFox | Veg | 1322.0 |
| 3 | 4.0 | 20-25 | -1 | LemonTree | Veg | 1234.0 |
| 4 | 5.0 | 35+ | 3 | Ibis | Vegetarian | 989.0 |
| 5 | 6.0 | 35+ | 3 | Ibys | Non-Veg | 1909.0 |
| 6 | 7.0 | 35+ | 4 | RedFox | Vegetarian | 1000.0 |
| 7 | 8.0 | 20-25 | 7 | LemonTree | Veg | 2999.0 |
| 8 | 9.0 | 25-30 | 2 | Ibis | Non-Veg | 3456.0 |
| 9 | 10.0 | 30-35 | 5 | RedFox | non-Veg | NaN |

| | NoOfPax | EstimatedSalary |
|---|---|---|
| 0 | 2.0 | 40000.0 |
| 1 | 3.0 | 59000.0 |
| 2 | 2.0 | 30000.0 |
| 3 | 2.0 | 120000.0 |
| 4 | 2.0 | 45000.0 |
| 5 | 2.0 | 122220.0 |
| 6 | NaN | 21122.0 |
| 7 | NaN | 345673.0 |

```
8       3.0                NaN
9       4.0           87777.0
```

```
df.Age_Group.unique()
```

```
array(['20-25', '30-35', '25-30', '35+'], dtype=object)
```

```
df.Hotel.unique()
```

```
array(['Ibis', 'LemonTree', 'RedFox', 'Ibys'], dtype=object)
```

```
df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
df.FoodPreference.unique
```

```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\456600217.py:1:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df.Hotel.replace(['Ibys'],'Ibis',inplace=True)
```

```
<bound method Series.unique of 0            veg
1       Non-Veg
2           Veg
3           Veg
4    Vegetarian
5       Non-Veg
6    Vegetarian
7           Veg
8       Non-Veg
9       non-Veg
Name: FoodPreference, dtype: object>
```

```
df.FoodPreference.replace(['Vegetarian','veg'],'Veg',inplace=True)
df.FoodPreference.replace(['non-Veg'],'Non-Veg',inplace=True)
```

```
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),inplace=Tru
e)
df.NoOfPax.fillna(round(df.NoOfPax.median()),inplace=True)
df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()),
inplace=True)
df.Bill.fillna(round(df.Bill.mean()),inplace=True)
df
```

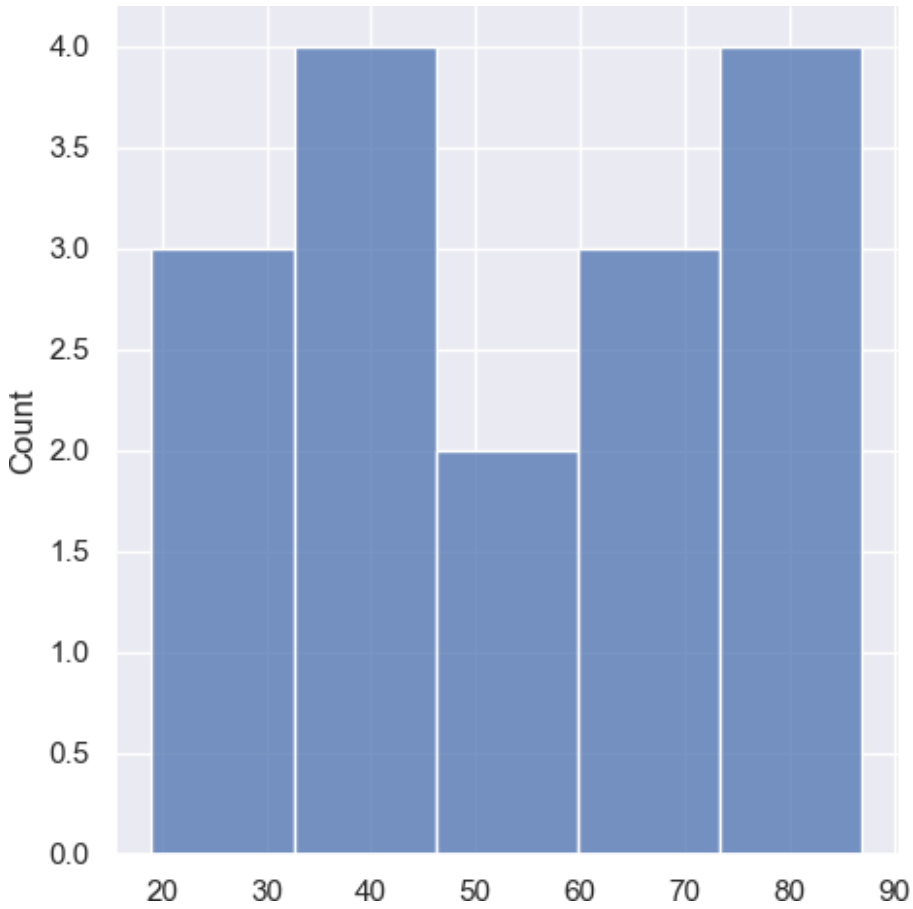
```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_67892\3711388855.py:3:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()),
inplace=True)
```

| | CustomerID | Age_Group | Rating(1-5) | Hotel | FoodPreference | Bill |
|---|---|---|---|---|---|---|
| 0 | 1.0 | 20-25 | 4 | Ibis | Veg | 1300.0 |
| 1 | 2.0 | 30-35 | 5 | LemonTree | Non-Veg | 2000.0 |
| 2 | 3.0 | 25-30 | 6 | RedFox | Veg | 1322.0 |
| 3 | 4.0 | 20-25 | -1 | LemonTree | Veg | 1234.0 |
| 4 | 5.0 | 35+ | 3 | Ibis | Veg | 989.0 |
| 5 | 6.0 | 35+ | 3 | Ibis | Non-Veg | 1909.0 |
| 6 | 7.0 | 35+ | 4 | RedFox | Veg | 1000.0 |
| 7 | 8.0 | 20-25 | 7 | LemonTree | Veg | 2999.0 |
| 8 | 9.0 | 25-30 | 2 | Ibis | Non-Veg | 3456.0 |
| 9 | 10.0 | 30-35 | 5 | RedFox | Non-Veg | 1801.0 |

| | NoOfPax | EstimatedSalary |
|---|---|---|
| 0 | 2.0 | 40000.0 |
| 1 | 3.0 | 59000.0 |
| 2 | 2.0 | 30000.0 |
| 3 | 2.0 | 120000.0 |
| 4 | 2.0 | 45000.0 |
| 5 | 2.0 | 122220.0 |
| 6 | 2.0 | 21122.0 |
| 7 | 2.0 | 345673.0 |
| 8 | 3.0 | 96755.0 |
| 9 | 4.0 | 87777.0 |

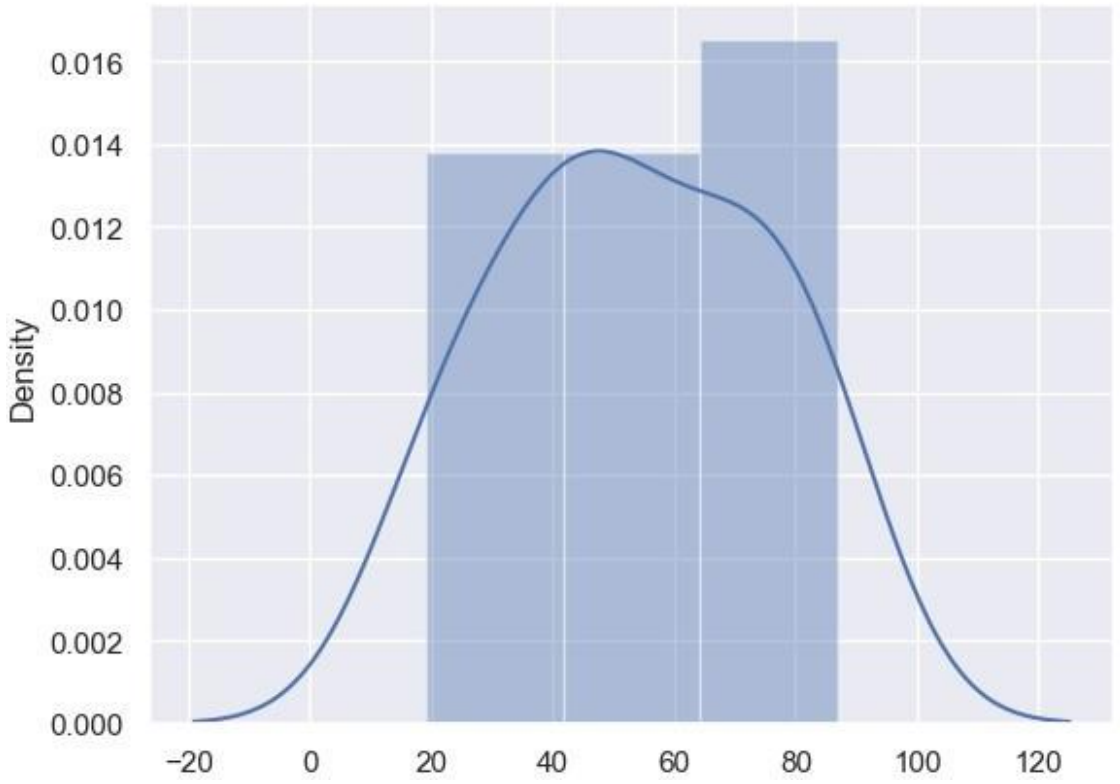
# EXERCISE 5

```
#sadha
#240701528
#19.8.25
#OutliersDetection

import numpy as np
array=np.random.randint(1,100,16)
array

array([63, 41, 75, 87, 83, 65, 45, 34, 81, 52, 46, 21, 70, 48, 19,
27],
      dtype=int32)

array.mean()

np.float64(53.5625)

np.percentile(array,25)

np.float64(39.25)

np.percentile(array,50)

np.float64(50.0)

np.percentile(array,75)

np.float64(71.25)

np.percentile(array,100)

np.float64(87.0)

def outDetection(array):
  sorted(array)
  Q1,Q3=np.percentile(array,[25,75])
  IQR=Q3-Q1
  lr=Q1-(1.5*IQR)
  ur=Q3+(1.5*IQR)
  return lr,ur
lr,ur=outDetection(array)
lr,ur

(np.float64(-8.75), np.float64(119.25))

import seaborn as sns
%matplotlib inline
sns.displot(array)
plt.show()
```




```
sns.distplot(array, kde=True)
plt.show()
```

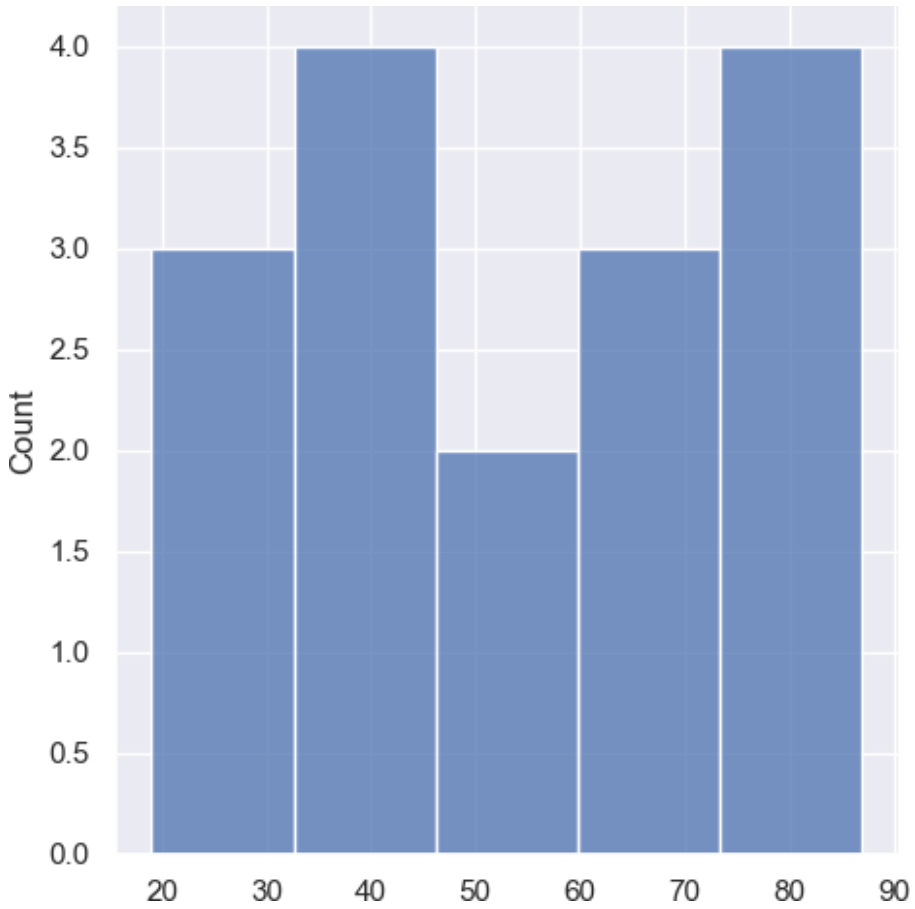
```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_55352\1660660772.py:1:
UserWarning:

`distplot` is a deprecated function and will be removed in seaborn
v0.14.0.

Please adapt your code to use either `displot` (a figure-level
function with
similar flexibility) or `histplot` (an axes-level function for
histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(array, kde=True)
```




```
new_array=array[(array>lr) & (array<ur)]
new_array
```

```
array([63, 41, 75, 87, 83, 65, 45, 34, 81, 52, 46, 21, 70, 48, 19,
27],
      dtype=int32)
```

```
sns.displot(new_array)
plt.show()
```




```
lr1,ur1=outDetection(new_array)
lr1,ur1
```

```
(np.float64(-8.75), np.float64(119.25))
```

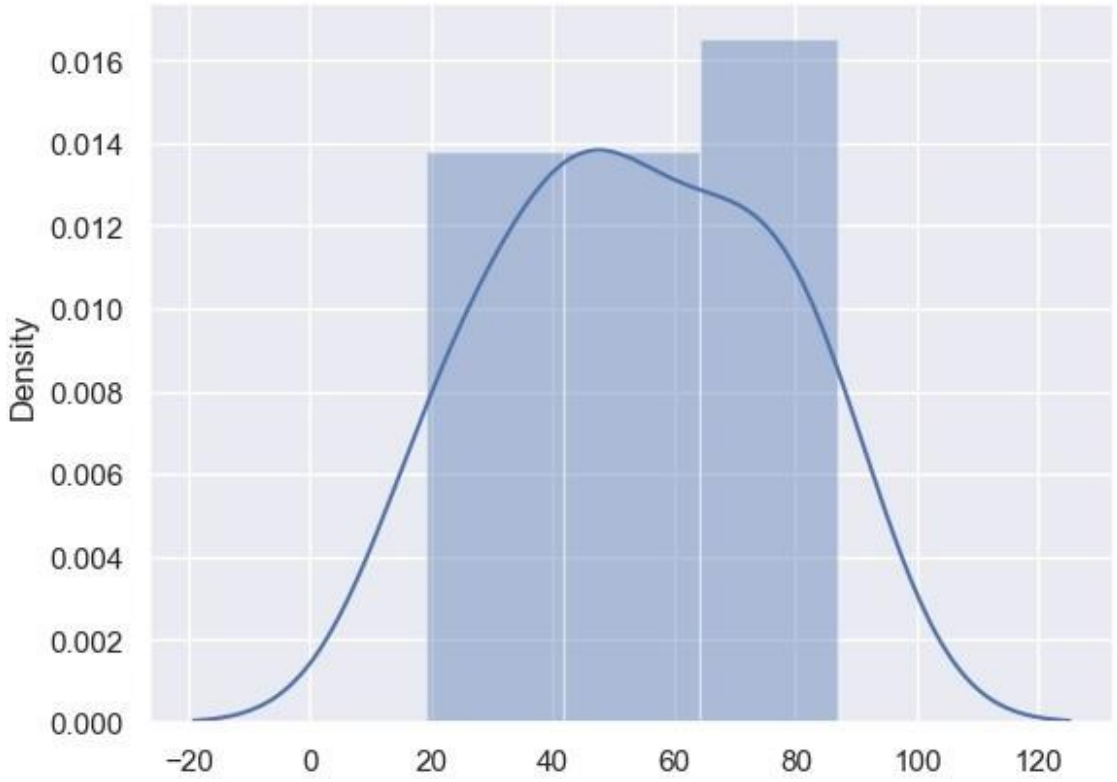
```
final_array=new_array[(new_array>lr1) & (new_array<ur1)]
final_array
```

```
array([63, 41, 75, 87, 83, 65, 45, 34, 81, 52, 46, 21, 70, 48, 19,
27],
      dtype=int32)
```

```
sns.distplot(final_array, kde=True)
plt.show()
```

```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_55352\3014262589.py:1:
UserWarning:

`distplot` is a deprecated function and will be removed in seaborn
v0.14.0.

Please adapt your code to use either `displot` (a figure-level
```

```
function with
similar flexibility) or `histplot` (an axes-level function for
histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(final_array, kde=True)
```



# EXERCISE 6

```
#sadha
#240701528
#19.8.25
#FeatureScaling

import numpy as np
import pandas as pd
df=pd.read_csv("C:\\Users\\sri sadha\\Downloads\\
pre_process_datasample.csv")
df
```

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8  Germany  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

```
df.head()
```

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
```

```
df.Country.fillna(df.Country.mode()[0],inplace=True)
features=df.iloc[:,:-1].values
```

```
C:\Users\sri sadha\AppData\Local\Temp\ipykernel_36908\3424832005.py:1:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df.Country.fillna(df.Country.mode()[0],inplace=True)
```

```
label=df.iloc[:,-1].values
from sklearn.impute import SimpleImputer
age=SimpleImputer(strategy="mean",missing_values=np.nan)
Salary=SimpleImputer(strategy="mean",missing_values=np.nan)
age.fit(features[:,[1]])
```

```
SimpleImputer()
```

```
Salary.fit(features[:,[2]])
```

```
SimpleImputer()
```

```
SimpleImputer()
```

```
SimpleImputer()
```

```
features[:,[1]]=age.transform(features[:,[1]])
features[:,[2]]=Salary.transform(features[:,[2]])
features
```

```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['Germany', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import OneHotEncoder
oh = OneHotEncoder(sparse_output=False)
Country=oh.fit_transform(features[:,[0]])
Country
```

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [0., 1., 0.],
       [1., 0., 0.]])
```

```
final_set=np.concatenate((Country,features[:,[1,2]]),axis=1)
final_set
```

```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
```

```
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [0.0, 1.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()
sc.fit(final_set)
feat_standard_scaler=sc.transform(final_set)
feat_standard_scaler
```

```
array([[ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-8.16496581e-01, -6.54653671e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [-8.16496581e-01,  1.52752523e+00, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.22474487e+00, -6.54653671e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
```

```
from sklearn.preprocessing import MinMaxScaler
mms=MinMaxScaler(feature_range=(0,1))
mms.fit(final_set)
feat_minmax_scaler=mms.transform(final_set)
feat_minmax_scaler
```

```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
```

```
       [0.        , 1.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

# EXERCISE 7

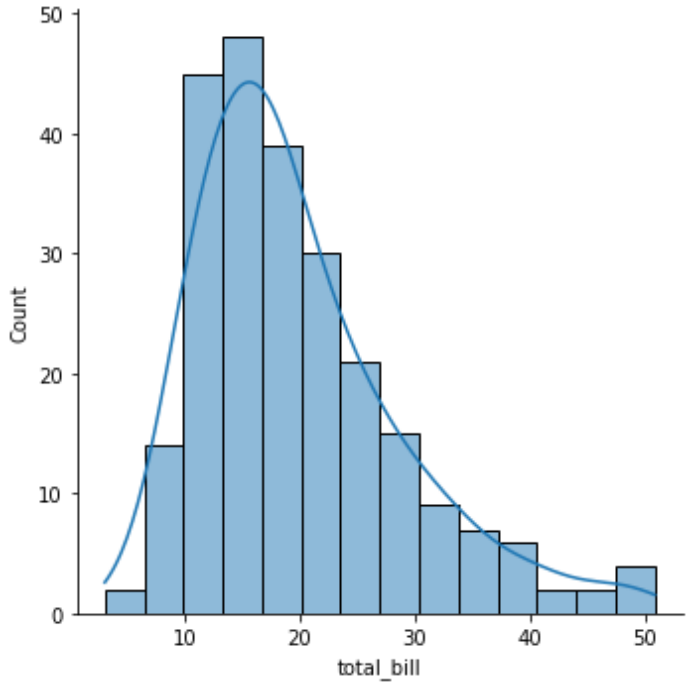
In [4]:
```
#sadha
#240701528
#26.8.25
#EDA
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
tips=sns.load_dataset('tips')
tips.head()
```

Out[4]:

| | total_bill | tip | sex | smoker | day | time | size |
|---|---|---|---|---|---|---|---|
| **0** | 16.99 | 1.01 | Female | No | Sun | Dinner | 2 |
| **1** | 10.34 | 1.66 | Male | No | Sun | Dinner | 3 |
| **2** | 21.01 | 3.50 | Male | No | Sun | Dinner | 3 |
| **3** | 23.68 | 3.31 | Male | No | Sun | Dinner | 2 |
| **4** | 24.59 | 3.61 | Female | No | Sun | Dinner | 4 |

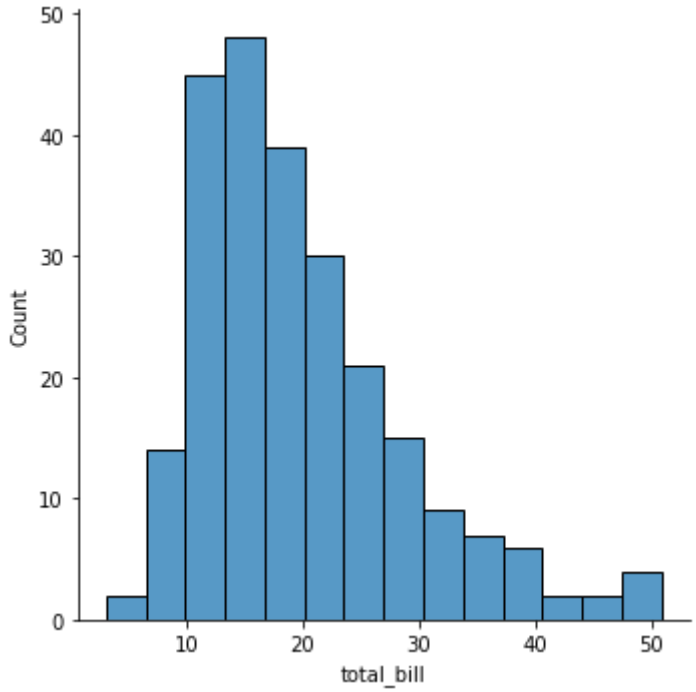
In [5]:
```
sns.displot(tips.total_bill,kde=True)
```

Out[5]:
```
<seaborn.axisgrid.FacetGrid at 0x275bd33da30>
```



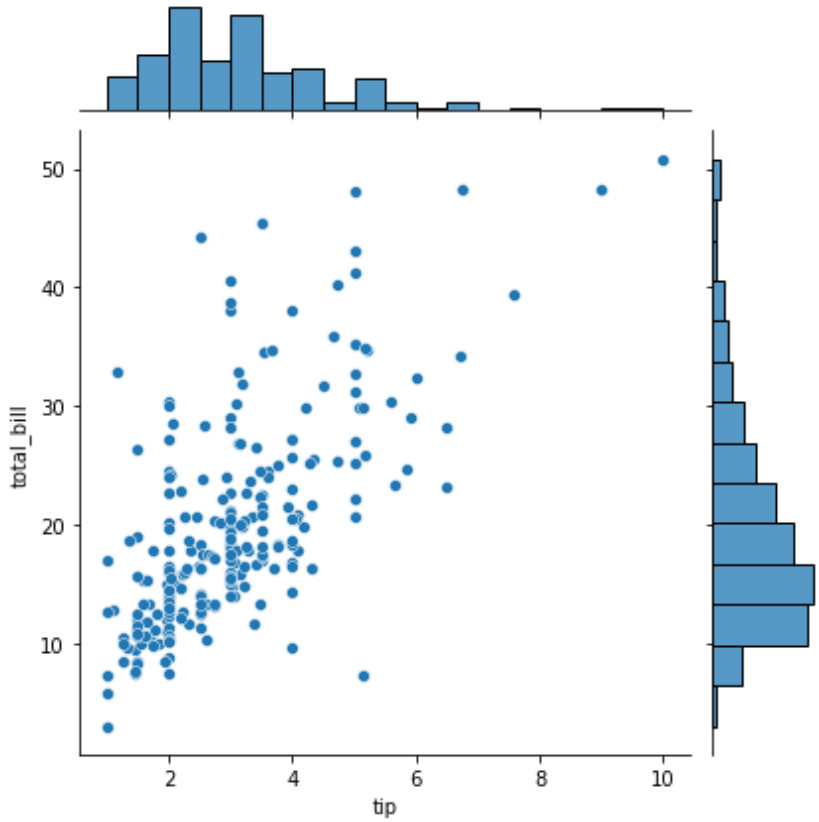
In [6]:
```
sns.displot(tips.total_bill,kde=False)
```

Out[6]:
```
<seaborn.axisgrid.FacetGrid at 0x275bdaa3250>
```



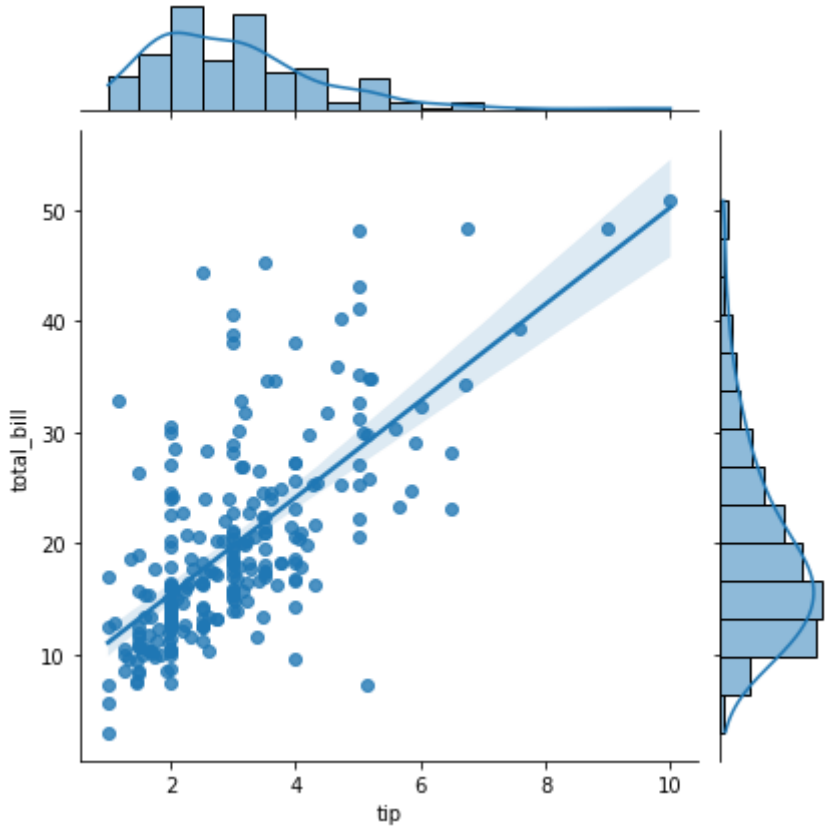

In [7]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill)
```

Out[7]:
```
<seaborn.axisgrid.JointGrid at 0x275b821e670>
```



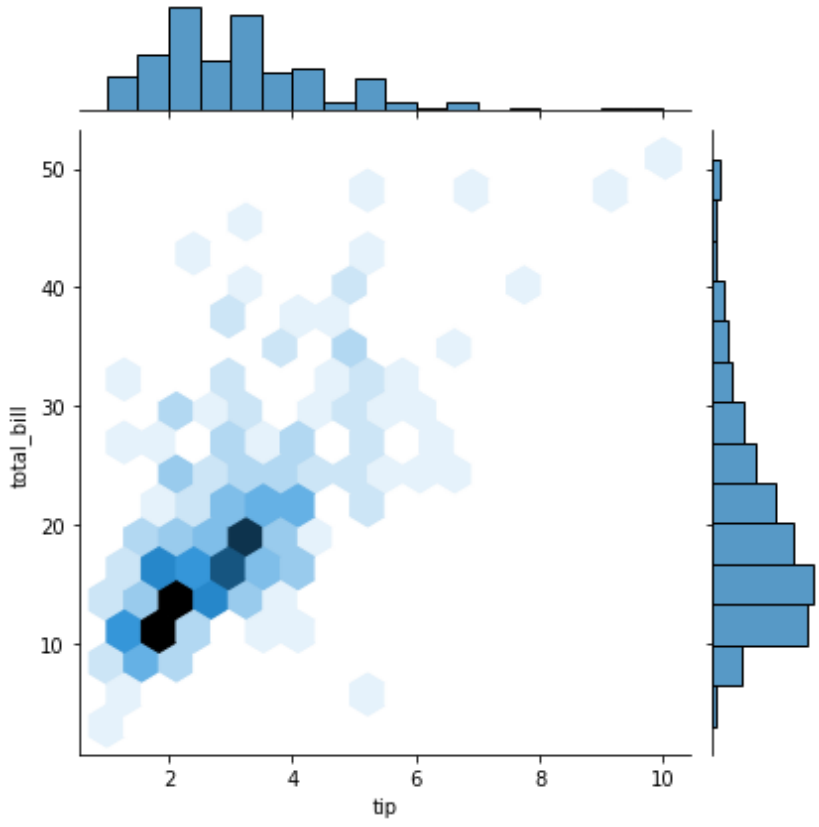
In [8]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="reg")
```

Out[8]:
```
<seaborn.axisgrid.JointGrid at 0x275be12f820>
```



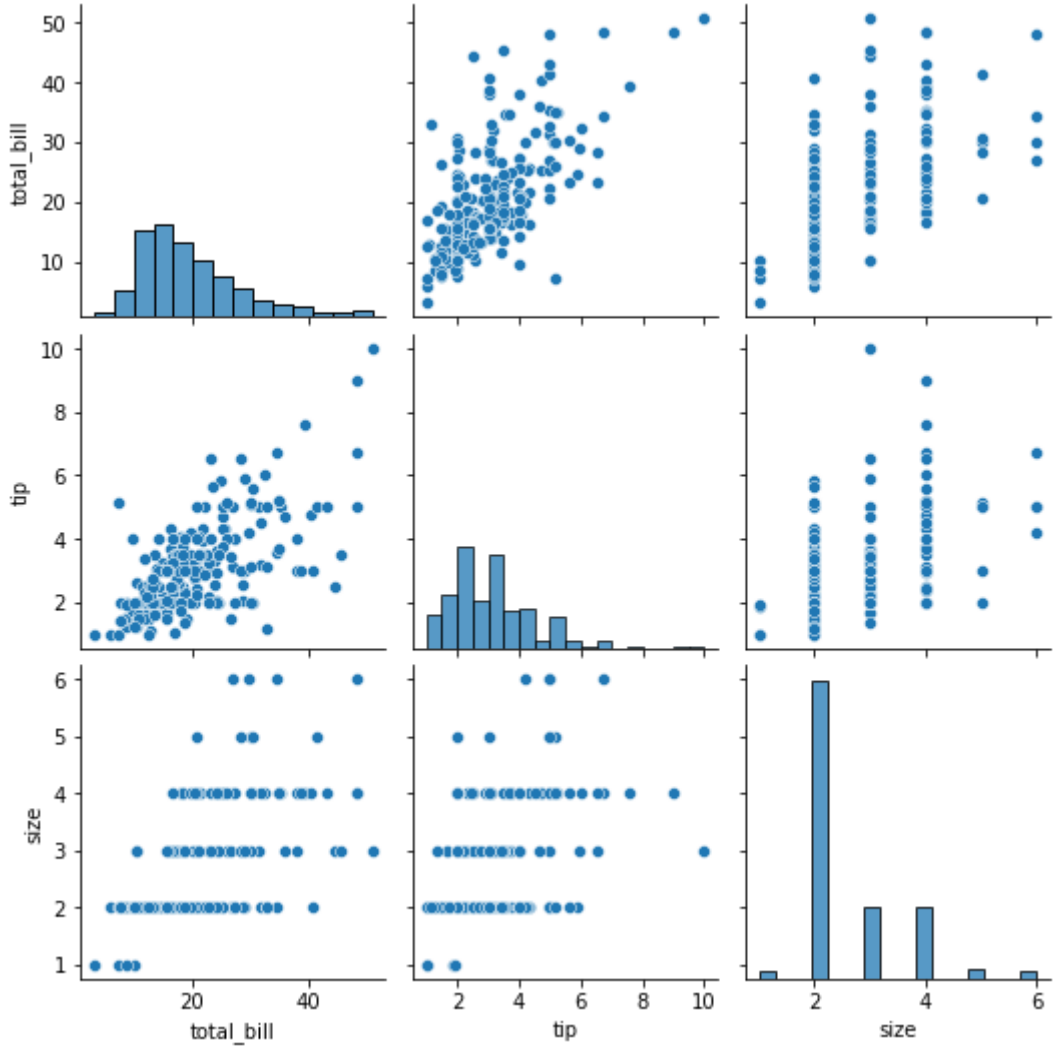

In [9]:
```
sns.jointplot(x=tips.tip,y=tips.total_bill,kind="hex")
```

Out[9]:
```
<seaborn.axisgrid.JointGrid at 0x275be1666d0>
```



In [10]:
```
sns.pairplot(tips)
```

Out[10]:
```
<seaborn.axisgrid.PairGrid at 0x275be424970>
```

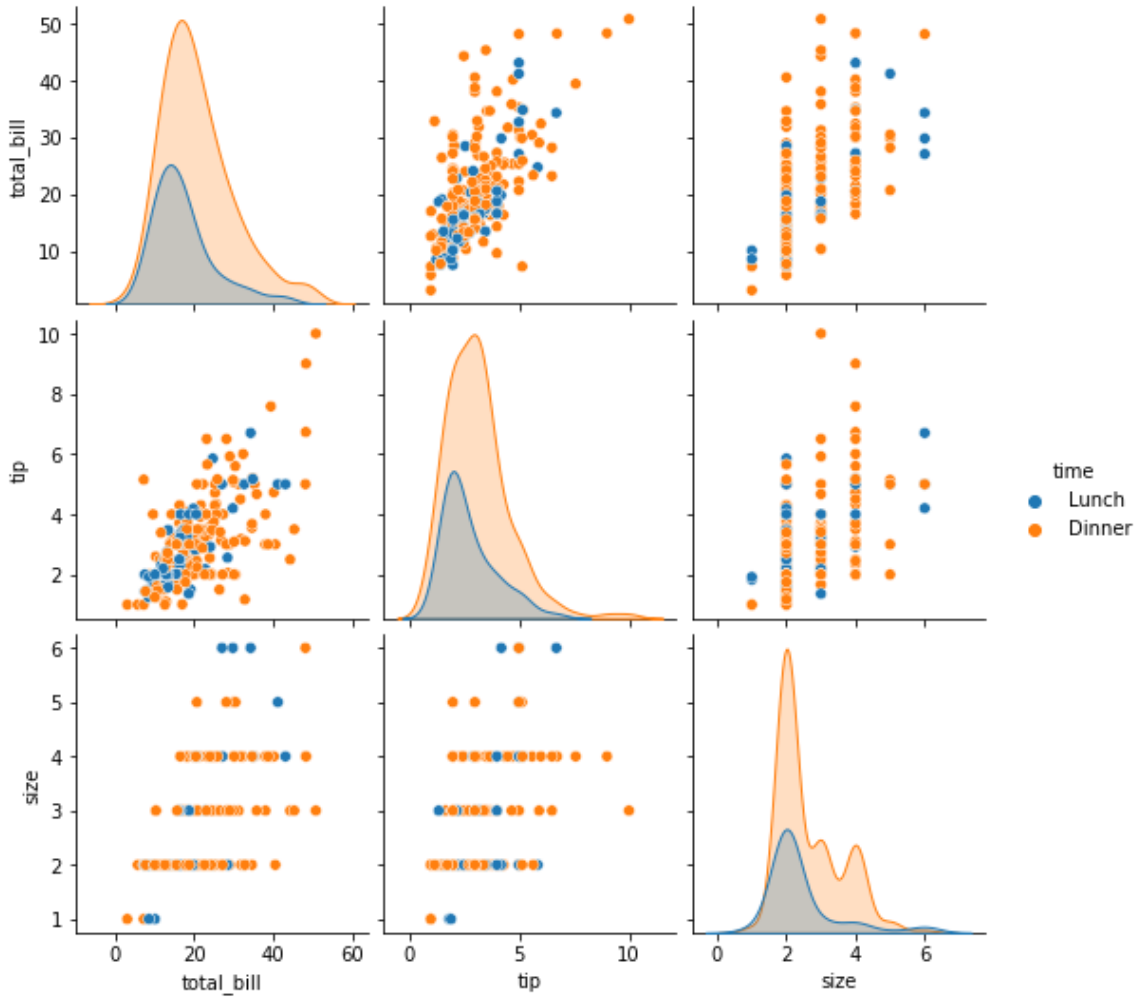



In [11]:
```
tips.time.value_counts()
```

Out[11]:
```
Dinner    176
Lunch      68
Name: time, dtype: int64
```

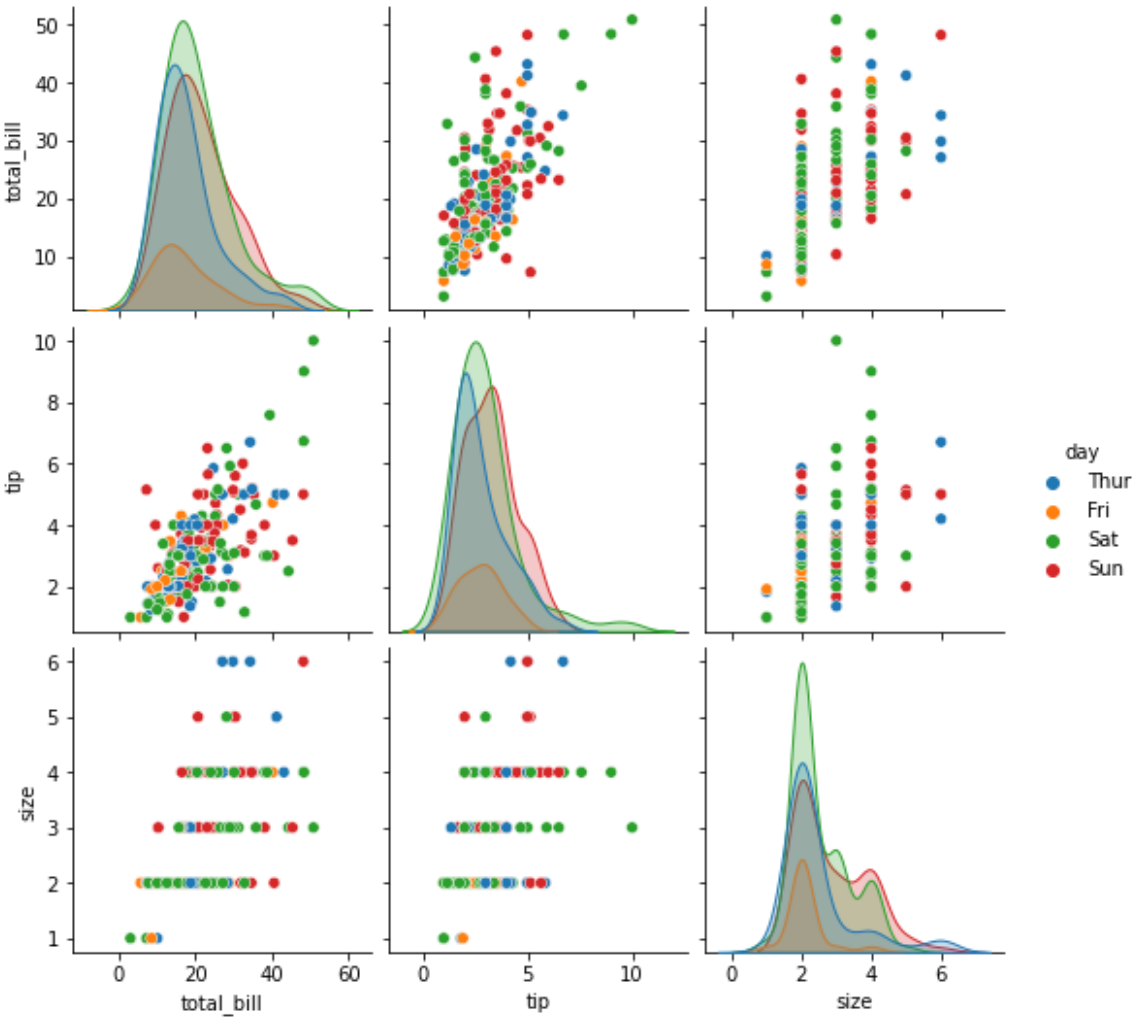
In [12]:
```
sns.pairplot(tips,hue='time')
```

Out[12]:
```
<seaborn.axisgrid.PairGrid at 0x275bf96e880>
```



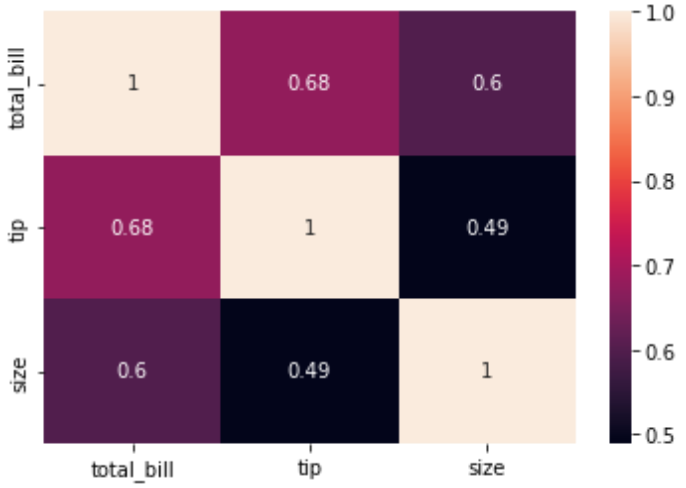
In [13]:
```
sns.pairplot(tips,hue='day')
```

Out[13]: <seaborn.axisgrid.PairGrid at 0x275bf8830a0>



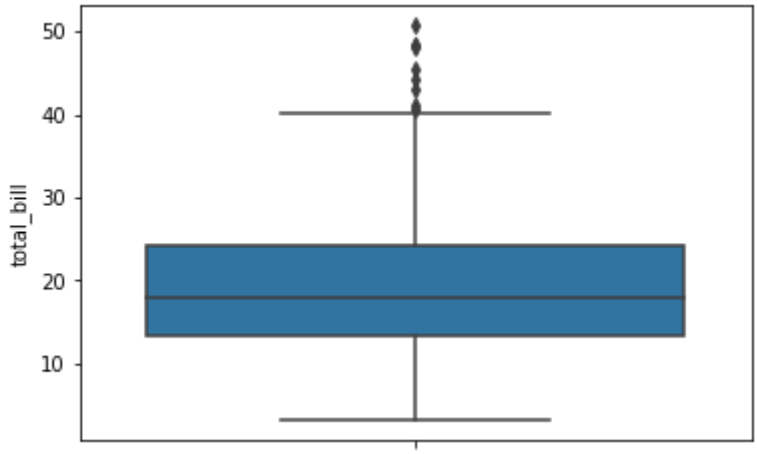

In [14]:
```
sns.heatmap(tips.corr(),annot=True)
```

Out[14]: <AxesSubplot:>



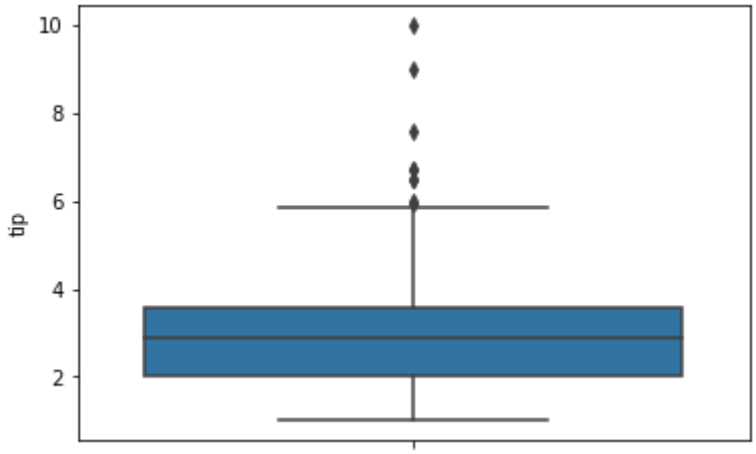
In [16]:
```
sns.boxplot(y=tips.total_bill)
```

Out[16]:
```
<AxesSubplot:ylabel='total_bill'>
```



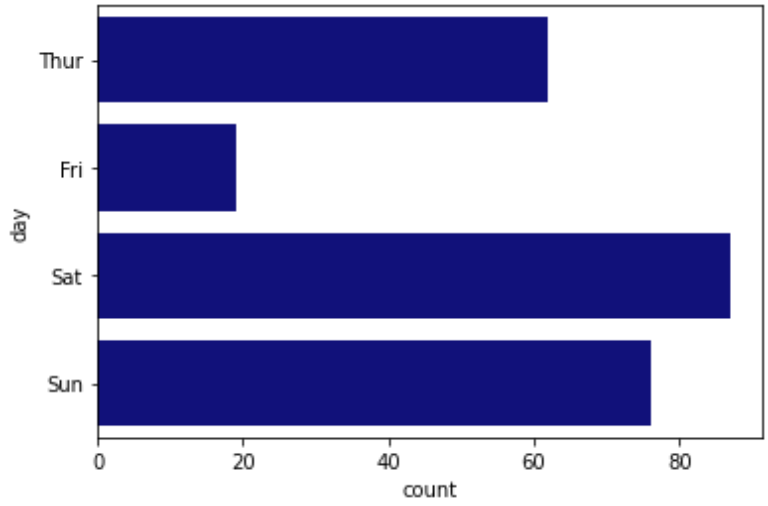

In [17]:
```
sns.boxplot(y=tips.tip)
```

Out[17]:
```
<AxesSubplot:ylabel='tip'>
```



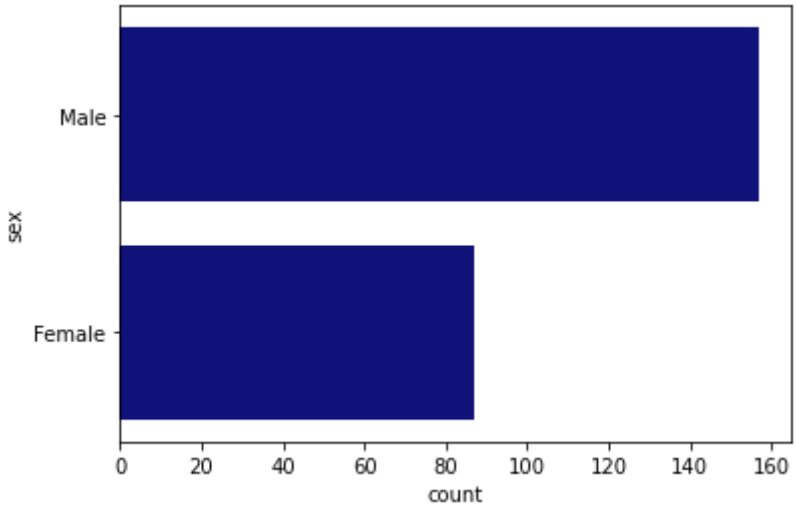

In [19]:
```
sns.countplot(y=tips.day,color='darkblue')
```

Out[19]:
```
<AxesSubplot:xlabel='count', ylabel='day'>
```



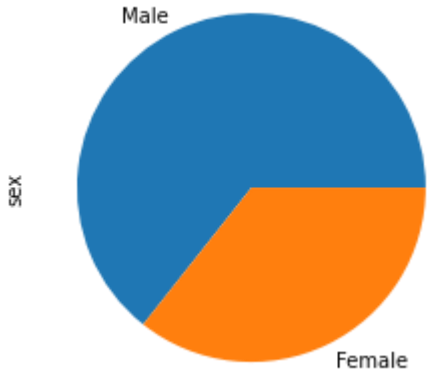
In [21]:
```
sns.countplot(y=tips.sex,color='darkblue')
```

Out[21]:
```
<AxesSubplot:xlabel='count', ylabel='sex'>
```



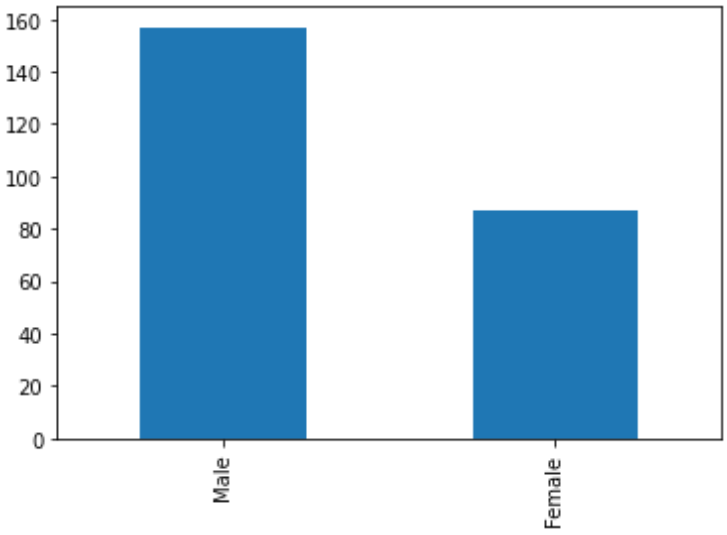

In [22]:
```
tips.sex.value_counts().plot(kind='pie')
```

Out[22]:
```
<AxesSubplot:ylabel='sex'>
```



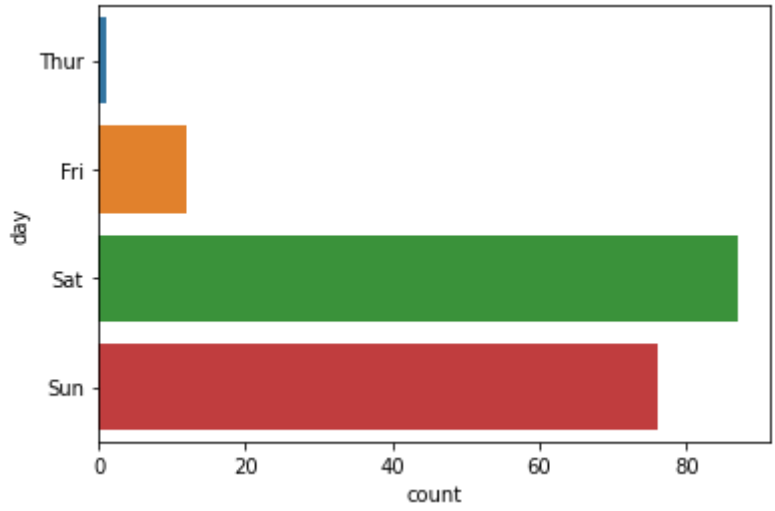

In [23]:
```
tips.sex.value_counts().plot(kind='bar')
```

Out[23]:
```
<AxesSubplot:>
```



In [25]:
```
sns.countplot(y=tips[tips.time=='Dinner']['day'])
```

Out[25]:
```
<AxesSubplot:xlabel='count', ylabel='day'>
```




In [ ]:
```

```

# EXERCISE 8

In [1]:
```
#sadha
#240701528
#Linear Regression
#23.9.25
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df
```

In [2]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 608.0 bytes
```

In [3]:
```
df.dropna(inplace=True)
```

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64  
dtypes: float64(1), int64(1)
memory usage: 608.0 bytes
```

In [5]:
```
df.describe()
```

Out[5]:

| | YearsExperience | Salary |
|---|---|---|
| **count** | 30.000000 | 30.000000 |
| **mean** | 5.313333 | 76003.000000 |
| **std** | 2.837888 | 27414.429785 |
| **min** | 1.100000 | 37731.000000 |
| **25%** | 3.200000 | 56720.750000 |
| **50%** | 4.700000 | 65237.000000 |
| **75%** | 7.700000 | 100544.750000 |
| **max** | 10.500000 | 122391.000000 |

In [6]:
```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values
```

In [7]:
```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=42
```

In [8]:
```
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

Out[8]:
```
LinearRegression()
```

In [9]:
```
model.score(x_train,y_train)
```

Out[9]:
```
0.9645401573418146
```

In [10]:
```
model.score(x_test,y_test)
```

Out[10]:
```
0.9024461774180497
```

In [11]:
```
model.coef_
```

Out[11]:
```
array([[9423.81532303]])
```

In [12]:
```
model.intercept_
```

Out[12]:
```
array([25321.58301178])
```

In [13]:
```
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))
```

In [14]:
```
model=pickle.load(open('SalaryPred.model','rb'))
```

In [15]:
```
yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
```

```
Enter Years of Experience: 42
```

In [16]:
```
print("Estimated Salary for {} years of experience is {}: " .format(yr_of_exp,Salary))
```

```
Estimated Salary for 42.0 years of experience is [[421121.82657908]]:
```

In [ ]:
```

```

# EXERCISE 9

In [17]:
```
#sadha
#240701528
#Logistic Regression
#7.10.25
import numpy as np
import pandas as pd
df=pd.read_csv('Social_Network_Ads.csv')
df
```

| | User ID | Gender | Age | EstimatedSalary | Purchased |
|---|---|---|---|---|---|
| **0** | 15624510 | Male | 19 | 19000 | 0 |
| **1** | 15810944 | Male | 35 | 20000 | 0 |
| **2** | 15668575 | Female | 26 | 43000 | 0 |
| **3** | 15603246 | Female | 27 | 57000 | 0 |
| **4** | 15804002 | Male | 19 | 76000 | 0 |
| **...** | ... | ... | ... | ... | ... |
| **395** | 15691863 | Female | 46 | 41000 | 1 |
| **396** | 15706071 | Male | 51 | 23000 | 1 |
| **397** | 15654296 | Female | 50 | 20000 | 1 |
| **398** | 15755018 | Male | 36 | 33000 | 0 |
| **399** | 15594041 | Female | 49 | 36000 | 1 |

400 rows × 5 columns

In [2]:
```
df.head()
```

Out[2]:

| | User ID | Gender | Age | EstimatedSalary | Purchased |
|---|---|---|---|---|---|
| **0** | 15624510 | Male | 19 | 19000 | 0 |
| **1** | 15810944 | Male | 35 | 20000 | 0 |
| **2** | 15668575 | Female | 26 | 43000 | 0 |
| **3** | 15603246 | Female | 27 | 57000 | 0 |
| **4** | 15804002 | Male | 19 | 76000 | 0 |

In [3]:
```
features=df.iloc[:,[2,3]].values
label=df.iloc[:,4].values
features
```

Out[3]:
```
array([[    19,  19000],
       [    35,  20000],
       [    26,  43000],
       [    27,  57000],
       [    19,  76000],
       [    27,  58000],
       [    27,  84000],
       [    32, 150000],
       [    25,  33000],
       [    35,  65000],
       [    26,  80000],
       [    26,  52000],
       [    20,  86000],
       [    32,  18000],
       [    18,  82000],
       [    29,  80000],
       [    47,  25000],
       [    45,  26000],
       [    46,  28000],
```

In [4]:
```
label
```

Out[4]:
```
array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1,
       0, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 0,
       1, 1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 1,
       0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       1, 1, 0, 1], dtype=int64)
```

In [6]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

In [11]:
```
for i in range(1,401):
  x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2,random_state=i
  model=LogisticRegression()
  model.fit(x_train,y_train)
  train_score=model.score(x_train,y_train)
  test_score=model.score(x_test,y_test)
  if test_score>train_score:
    print("Test {} Train{} Random State {}".format(test_score,train_score,i))
```

```
Test 0.6875 Train0.63125 Random State 3
Test 0.7375 Train0.61875 Random State 4
Test 0.6625 Train0.6375 Random State 5
Test 0.65 Train0.640625 Random State 6
Test 0.675 Train0.634375 Random State 7
Test 0.675 Train0.634375 Random State 8
Test 0.65 Train0.640625 Random State 10
Test 0.6625 Train0.6375 Random State 11
Test 0.7125 Train0.625 Random State 13
Test 0.675 Train0.634375 Random State 16
Test 0.7 Train0.628125 Random State 17
Test 0.7 Train0.628125 Random State 21
Test 0.65 Train0.640625 Random State 24
Test 0.6625 Train0.6375 Random State 25
Test 0.75 Train0.615625 Random State 26
Test 0.675 Train0.634375 Random State 27
Test 0.7 Train0.628125 Random State 28
Test 0.6875 Train0.63125 Random State 29
Test 0.6875 Train0.63125 Random State 31
```

In [14]:
```
x_train,x_test,y_train,y_test=train_test_split(features,label,test_size=0.2)
finalModel=LogisticRegression()
finalModel.fit(x_train,y_train)
```

Out[14]:
```
LogisticRegression()
```

In [15]:
```
print(finalModel.score(x_train,y_train))
print(finalModel.score(x_test,y_test))
```

```
0.8625
0.825
```

In [16]:
```
from sklearn.metrics import classification_report
print(classification_report(label,finalModel.predict(features)))
```

```
              precision    recall  f1-score   support

           0       0.86      0.92      0.89       257
           1       0.83      0.74      0.79       143

    accuracy                           0.85       400
   macro avg       0.85      0.83      0.84       400
weighted avg       0.85      0.85      0.85       400
```

In [ ]:
```

```

# EXERCISE 10

In [1]:
```
#sadha
#240701528
#K_Means_Clustering
#7.10.25
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

In [2]:
```
df=pd.read_csv('Mall_Customers.csv')
```

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype 
---  ------                  --------------  ----- 
 0   CustomerID              200 non-null    int64 
 1   Gender                  200 non-null    object
 2   Age                     200 non-null    int64 
 3   Annual Income (k$)      200 non-null    int64 
 4   Spending Score (1-100)  200 non-null    int64 
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

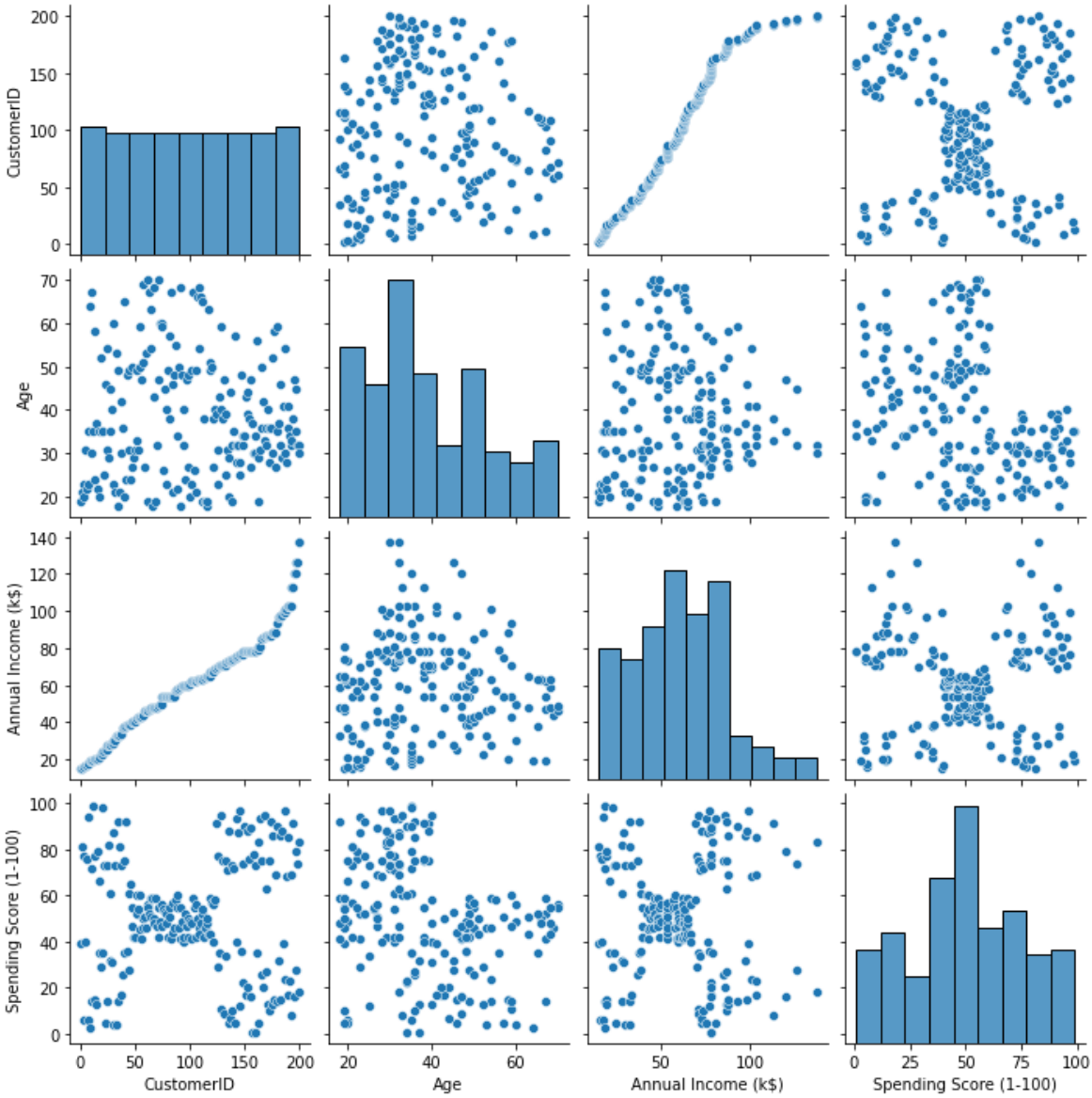
In [4]:
```
df.head()
```

Out[4]:

| | CustomerID | Gender | Age | Annual Income (k$) | Spending Score (1-100) |
|---|---|---|---|---|---|
| **0** | 1 | Male | 19 | 15 | 39 |
| **1** | 2 | Male | 21 | 15 | 81 |
| **2** | 3 | Female | 20 | 16 | 6 |
| **3** | 4 | Female | 23 | 16 | 77 |
| **4** | 5 | Female | 31 | 17 | 40 |

In [5]:
```
sns.pairplot(df)
```

Out[5]:
```
<seaborn.axisgrid.PairGrid at 0x1e020965730>
```




In [6]:
```
features=df.iloc[:,[3,4]].values
```

In [7]:
```
from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)
```

Out[7]:
```
KMeans(n_clusters=5)
```

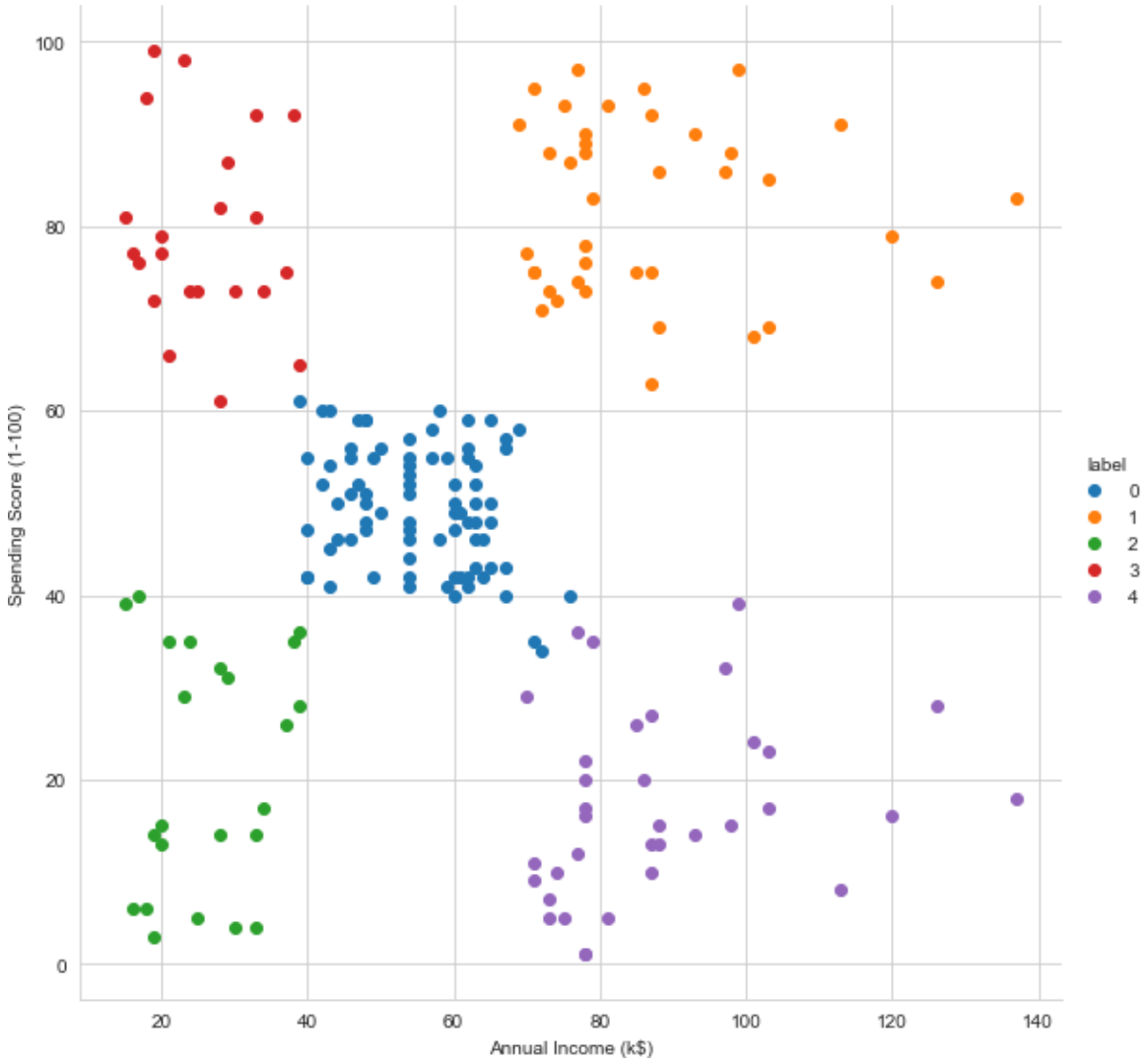
In [8]:
```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

```
C:\Users\hdc0422189\AppData\Local\Temp\ipykernel_6788\470183701.py:2: Settin
gWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/
stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pand
as.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-v
ersus-a-copy)
  Final['label']=model.predict(features)
```

Out[8]:

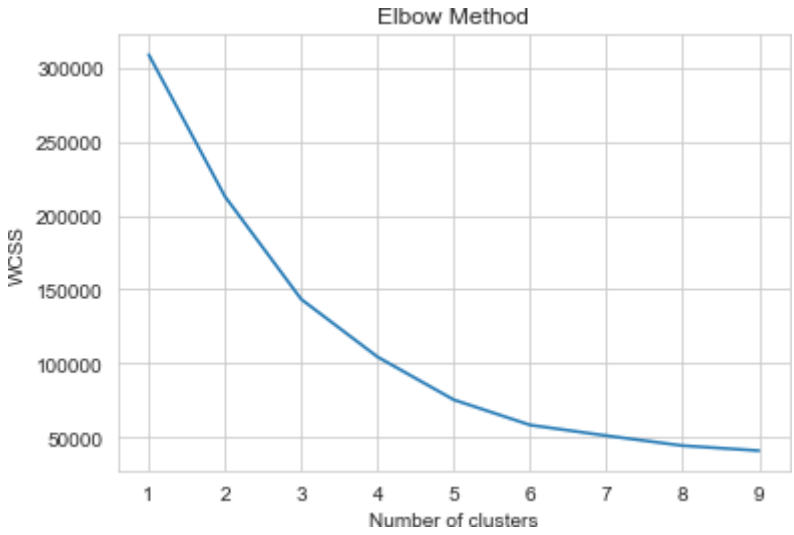
| | Annual Income (k$) | Spending Score (1-100) | label |
|---|---|---|---|
| **0** | 15 | 39 | 2 |
| **1** | 15 | 81 | 3 |
| **2** | 16 | 6 | 2 |
| **3** | 16 | 77 | 3 |
| **4** | 17 | 40 | 2 |

In [9]:
```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```



In [14]:
```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
features_el = df.iloc[:, [2, 3, 4]].values
wcss = []
for i in range(1, 10):
    model = KMeans(n_clusters=i, random_state=0)
    model.fit(features_el)
    wcss.append(model.inertia_)
plt.plot(range(1, 10), wcss)
plt.title('Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```

```
C:\ProgramData\Anaconda3\lib\site-packages\sklearn\cluster\_kmeans.py:1036:
UserWarning: KMeans is known to have a memory leak on Windows with MKL, when
there are less chunks than available threads. You can avoid it by setting th
e environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```




In [ ]:

# EXERCISE 11

In [6]:
```
#sadha
#240701528
#KNN
#7.10.25
import numpy as np
import pandas as pd
df=pd.read_csv('Iris.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   sepal.length   150 non-null    float64
 1   sepal.width    150 non-null    float64
 2   petal.length   150 non-null    float64
 3   petal.width    150 non-null    float64
 4   variety        150 non-null    object 
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

In [7]:
```
df.variety.value_counts()
```

Out[7]:
```
Setosa         50
Versicolor     50
Virginica      50
Name: variety, dtype: int64
```

In [8]:
```
df.head()
```

Out[8]:

| | sepal.length | sepal.width | petal.length | petal.width | variety |
|---|---|---|---|---|---|
| **0** | 5.1 | 3.5 | 1.4 | 0.2 | Setosa |
| **1** | 4.9 | 3.0 | 1.4 | 0.2 | Setosa |
| **2** | 4.7 | 3.2 | 1.3 | 0.2 | Setosa |
| **3** | 4.6 | 3.1 | 1.5 | 0.2 | Setosa |
| **4** | 5.0 | 3.6 | 1.4 | 0.2 | Setosa |

In [9]:
```
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values
```

In [10]:
```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

In [12]:
```
xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2,random_state=0
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)
```

Out[12]:
```
KNeighborsClassifier()
```

In [13]:
```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
```

```
0.95
0.9666666666666667
```

In [14]:
```
from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))
```

Out[14]:
```
array([[50,  0,  0],
       [ 0, 45,  5],
       [ 0,  2, 48]], dtype=int64)
```

In [15]:
```
from sklearn.metrics import classification_report
print(classification_report(label,model_KNN.predict(features)))
```

```
              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.96      0.90      0.93        50
   Virginica       0.91      0.96      0.93        50

    accuracy                           0.95       150
   macro avg       0.95      0.95      0.95       150
weighted avg       0.95      0.95      0.95       150
```

In [ ]:

# EXERCISE 12

[2]:
```
#sadha
#240701528
#28.10.25
#T_test
```

[3]:
```
import numpy as np
from scipy import stats
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
mu_0 = 70
t_stat, p_value = stats.ttest_1samp(marks, mu_0)
print(f"T-statistic: {t_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value<alpha:
  print("Reject Null Hypothesis → Mean is significantly different from 70.")
else:
  print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
T-statistic: 1.993
P-value: 0.0774
Fail to Reject Null Hypothesis → No significant difference.
```

[ ]:

# EXERCISE 13

[4]:
```
#sadha
#240701528
#28.10.25
#Z_test
```

[6]:
```
import numpy as np
from math import sqrt
from scipy.stats import norm
# Given data
x_bar = 51.2
mu_0 = 50
sigma = 3
n = 36
z_stat = (x_bar - mu_0) / (sigma / sqrt(n))
p_value = 2 * (1 - norm.cdf(abs(z_stat)))
print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value<alpha:
  print("Reject Null Hypothesis → Mean is significantly different from 50 g.")
else:
  print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
Z-statistic: 2.400
P-value: 0.0164
Reject Null Hypothesis → Mean is significantly different from 50 g.
```

[ ]:

# EXERCISE 14

[8]:
```
#sadha
#240701528
#28.10.25
#AnovaTest
```

[9]:
```
import numpy as np
from scipy import stats
# Data
A = [20, 22, 23]
B = [19, 20, 18]
C = [25, 27, 26]
f_stat, p_value = stats.f_oneway(A, B, C)
print(f"F-statistic: {f_stat:.3f}")
print(f"P-value: {p_value:.4f}")
alpha = 0.05
if p_value<alpha:
  print("Reject Null Hypothesis → Means are significantly different.")
else:
  print("Fail to Reject Null Hypothesis → No significant difference.")
```

```
F-statistic: 25.923
P-value: 0.0011
Reject Null Hypothesis → Means are significantly different.
```

[ ]: